Journal of Network and Systems Management (2026) 34:34
https://doi.org/10.1007/s10922-025-10005-6

S-iNAS: A Performance-Centric SRN Modeling and Analysis of Time and Event Based Scaling Strategies in Ceph-Based Industrial Network-Attached Storage

Damsub Lim[1] · Tuan Anh Nguyen[2] · Jueying Li[1] · Eunmi Choi[3] · Dugki Min[1,4] · Ruben Machucho Cadena[5] · Bruno Nogueira[6] · Ermeson Andrade[7] · Gustavo Callou[7] · Mario Di Mauro[8] · Matheus Torquato[9,10] · Francisco Airton Silva[11] · Paulo Maciel[12]

Received: 8 March 2025 / Revised: 13 July 2025 / Accepted: 29 October 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

Emerging Industry 4.0 applications and digital twin paradigms generate massive, highly dynamic data streams that pose significant challenges for storage infrastructures in industrial environments. In this study, we address the issue of scalable performance by introducing S-iNAS, a scalable Ceph-based framework capable of adapting to sharply fluctuating workloads while maintaining high throughput and low latency for industrial network-attached storages (iNAS). Our approach leverages a SRN model to rigorously capture key performance factors: concurrent read/write workflows, replication overhead, and dynamic scaling triggers for Ceph Object Storage Daemons (OSDs). We investigate two principal Scaling Policies—time-based (scheduled expansions) and event-based (threshold-triggered)—under varied arrival rates, client concurrency levels, VM instantiation delays, and read/write compositions. Experimental results from extensive Sensitivity Analyses demonstrate that time-based scaling, although maintaining throughput stability under moderate loads, risks delayed responsiveness when demands spike. By contrast, event-based scaling offers agile adaptation and mitigates latency at the cost of more frequent reconfigurations. These findings highlight how even small modifications to scale-out thresholds or scheduling intervals can substantially influence performance. Compared with existing Petri Net or autoscaling research, our SRN driven modelling helps practitioners comprehend Ceph's replication and I/O concurrency behaviors, thereby filling a critical gap in performance-centric solutions for iNAS. Ultimately, S-iNAS equips practitioners with actionable guidelines to fine-tune scal-

Damsub Lim and Tuan Anh Nguyen have contributed equally to this work.

Extended author information available on the last page of the article

Published online: 05 December 2025

ing policies, ensuring robust, cost-effective, and high-throughput data services for mission-critical industrial workflows and digital twin ecosystems.

Keywords Stochastic reward net · Ceph · Auto-scaling · Industrial NAS · Performability

1 Introduction

Modern industrial infrastructures are witnessing an exponential surge in data volume, fueled by the proliferation of sensors, real-time monitoring devices, and interconnected machinery—hallmarks of Industry4.0 [1, 2]. Simultaneously, the drive toward digital transformation continues to reshape enterprise operations, resulting in mounting demands for rapid data access, high-throughput analytics, and minimal response times [3]. In such contexts, Scalable Network Attached Storage (S-NAS) systems have emerged as indispensable solutions for modern IT infrastructures, offering the capacity to handle large datasets and the flexibility to adapt to diverse industrial or enterprise settings. By centralizing data management, reinforcing data protection, and ensuring enhanced accessibility, S-NAS platforms have established themselves as fundamental building blocks for data-centric applications in cloud services, large-scale enterprises, and industrial sectors [4]. Despite these strengths, S-NAS deployments are seldom straightforward due to their inherently large scale, varied workloads, and intricate interplay of hardware and software components. In particular, conventional NAS solutions may require manual reconfiguration or hardware upgrades to adapt to changing demand, leading to periods of [5]. Consequently, designing an industrial-grade NAS capable of dynamically adjusting resources to maintain consistent high throughput and low latency across fluctuating workloads poses a significant challenge. In particular, the adoption of S-NAS in industrial sectors can maximize the efficiency of data storage and processing, providing opportunities to enhance the performance of real-time system analysis and decision support systems by utilizing stored data. For instance, manufacturers can collect and analyze large volumes of sensor data through S-NAS to optimize production processes, while the logistics industry can use S-NAS to improve inventory management and supply chain efficiency. These application cases further underscore the necessity of S-NAS, as the real-time accessibility and security of the data obtained are crucial elements in strengthening competitive advantage. However, the complexity of these S-NAS systems can lead to issues such as performance degradation, which may ultimately negatively impact the operational efficiency of S-NAS. Therefore, employing the SRN model for performance measurement will play an important role in identifying potential issues within S-NAS systems in advance and establishing optimal configuration and operational strategies. This approach will enable organizations to maintain predictable performance and build a foundation for agile responses to changing requirements. A promising candidate to address these challenges is the open-source, highly configurable Ceph architecture. Ceph's cluster-based design, featuring Object Storage Daemons (OSDs), the Controlled Replication Under Scalable Hashing (CRUSH) algorithm for data distribution, and optional support for block, file, or object back-

ends, aligns strongly with industrial environments where robust fault tolerance, concurrent I/O operations, and seamless rebalancing are key [6]. However, while Ceph is inherently scalable, effectively leveraging its scale-out model in real-time production settings calls for deeper insights into how performance metrics (e.g., throughput and latency) respond to dynamic workload conditions and resource expansions.

Motivation for network and systems management: Expanding Industry 4.0 deployments and the *digital twin* concept push industrial Network Attached Storage (iNAS) to sustain high perfomance for swinging data loads. We offer a *performance-centric* analitical framework built on *SRN* for Ceph backed iNAS that evaluates time based and event based autoscaling. The formal model unifies perfomance and dependability into a single performability view, emphasising througput and latency, its predictions can guide administrators in provisioning resources and choosing dynamic policies, thus manual rule sets become unecessary under every load scenario.

From the viewpoint of network and systems management, S-iNAS spans several important topics. (i), it tackles *performance modelling with SRN* for distributed storage systems. SRN extends Petri nets through stochastic transitions and rewards, enabling parallel analys of performance and reliability [7]. Applying SRN to Ceph iNAS builds a formal model that records througput and latency for diverse load states, pushing beyond earlier cloud availability studies [8]. (ii) The study reviews *time based versus event based autoscaling strategies*. Schedule based autoscaling invokes changes at fixed intervals whereas threshold triggered autoscaling reacts to real time metrics. This issue relates to automated network service management, covering Network Functions Virtualization (NFV) controllers and cloud allocation with schemes ranging from static rules and feedback control to reinforcement learning [9, 10]. (iii) The work addresses *distributed storage cluster management and resource orchestration*. Ceph provides scale out capability via OSDs and the CRUSH algorithm [11]. Industrial adoption demands dynamic orchestration, for example adding or removing OSDs when load varies while balancing performance with fault tolerance. The S-iNAS SRN model includes read, write, data replication, plus dynamic OSD operations, letting analysts inspect how cluster scale alters latency and througput. The analysis further covers *system monitoring and telemetry*. Event driven autoscaling requires that monitoring suports reliable capture of queue length and real time utilisation, making streaming telemetry a preferred option over legacy Simple Network Management Protocol (SNMP). Finally, (iv) S-iNAS links to *intent based resource management and AI driven dynamic orchestration*. Autoscaling rules such as "expand if the queue exceeds 50%" serve as high level intents that hold performance within bounds, matching zero touch service management in 5 G and 6 G [12, 13]. Taken together, this work positions itself in network and service operation and management, coupling quantitative SRN modelling, storage system performance control, and a path toward fully automated intent driven architectures.

S-iNAS and allied studies flag hurdles in autoscaling distributed storage for industry. A chief concern is *guaranteeing performability, that is, sustaining both high performance and reliability, when the system scales dynamically*, a target weakened once Ceph inserts or removes OSDs, disrupting data balance and forcing costly rebalancing [14, 15]. Measurements confirm throughput rises while latency raise because data migrate [16]. The S-iNAS model simplifed finite capacity and rebalance time;

therefore *fully modeling the complex behavior of a real system* is extremely challeng and SRN or Petri net analist still face state space explosion. *Selecting an autoscaling strategy that matches the application load* is the next obstacle. Time based rules look steady, nonetheless they risk *slow reaction* during bursts, queues build, latency climbs [17, 18]. Event driven scaling trims latency during abrupt demand; However careless thresholds provoke oscillation or idle resources, leaving the balance subtle. Cloud work groups autoscaling into *static threshold reaction*, *schedule based*, and closed loop or machine learning prediction, each limited [3, 19]. Analysts must *determine optimal thresholds or schedules for autoscaling*. S-iNAS tested 60, 120, and 180 s periods alongside event thresholds of 25%, 50%, and 75% [20, 21]. A 25% threshold delivered the quickest reaction, higher throughput, and lower latency relative to the other settings [22]; however excessive sensitivity triggers *too many* events, wasting resources or causing oscillation. Parameter optimisation stays crucial across varied read or write workloads. *Maintaining system adaptability in real industrial settings where workloads are highly diverse and irregular* remains open since S-iNAS assumed homogeneous 1GB requests cut into 4MB chunks and ignored OSD capacity limits [23, 24].

In real enviroments data sizes and access profiles span tiny logs to high resolution video [24–26]; when an OSD nears capacity perfomance drops and Ceph rebalances, incurring heavy cost. Such dynamics resist concise analytic models. Hence the task is to *integrate workload characteristics and resource constraints into the automatic orchestration mechanism*. AI driven schemes that detect load patterns and retune autoscaling in real time exist [27–29], however training demands vast monitoring traces and low latency feedback, so telemetry and analytics stay primary bottlenecks. Another obligation is *ensuring fault tolerance and reliability during automated orchestration*. Ceph keeps replicas across many OSDs [6, 30]. Adding or removing devices must respect the replication factor, therefore autoscaling initiates extended copy operations that need coordination with fault handling, for instance when an OSD crashes while expansion proceeds. SRN based studies already quantify the *trade off* between throughput and reliability in replicated storage [31, 32]. A realistc smart orchestrator hence couples autoscale with health monitors and remediation, such as pausing scaling when failures arise, to hold stability. Resource orchestration for Ceph still needs predictive models, scenario tuned policies, prompt trustworthy telemetry, and fault aware coordination.

Motivation for a Performance-Centric SRN Approach Traditional performance evaluation methods for large-scale storage systems—such as queueing theory or simplified Markov models—often fail to capture the stochastic, parallel, and multi-stage nature of Ceph processes [33]. Moreover, existing studies on autoscaling frequently target virtual machines [12, 13] or container-based applications [34] rather than storage-layer logic. This limitation highlights a critical gap [35, 36]: How can one systematically analyze the impact of time-based vs. event-based scaling on throughput and response time in a Ceph-based NAS, especially under rapidly fluctuating industrial workloads?

The Petri Nets (PN) is the formal description of the system dynamic behavior with Place, Timed Transitions, Tokens and Arcs. The Stochastic Petri Nets (SPN) applies event and time information to the Markov process in relation to all the indications of

the reachability graph of the PN. The Generalized Stochastic Petri Nets (GSPN) is an extended model of the SPN model, and it is a model with Immediate Transitions removed from the Reachability Graph and not added to the Markov process, and The inhibitor Arcs that prevent the behavior of a specific transition when a token is located in the Place. SRN provide a useful high-level language for evaluating large systems, representation of the intricate behavior of such systems often leads to large and complex structure of the GSPN. It allows for simultaneous performance and reward analysis with the variable cardinality arc, priorities, guard functions [37, 38].

This gap motivates the use of SRNs, an extension of Petri nets that integrates probabilistic transitions and reward mechanisms. SRNs excel in modeling random and potentially disruptive events such as spikes in sensor data, hardware bottlenecks, and irregular arrival patterns—all central to industrial data storage. By representing system states, transitions, and resource usage in detail, SRNs allow for in-depth investigations of performance metrics like throughput and latency (rather than focusing on reliability or availability) [16]. This performance-first viewpoint is paramount in industrial scenarios, where any sudden bottleneck or elongated latency can disrupt mission-critical services such as real-time monitoring, predictive maintenance, or safety management [14].

Research Focus and Novelty Building upon these core strengths, our study adopts an SRN-based approach to evaluate and optimize Scalability in Industrial Network Attached Storage Systems (S-iNAS). Specifically, we model two distinct Scaling Policies—Time-based (fixed intervals) and Event-based (threshold-triggered)—within a Ceph-based S-NAS framework. Our objective is to quantify how these policies influence key performance metrics under varying workloads, client concurrency, and VM instantiation delays. Time-based scaling provides stable performance in industrial environments with predictable loads because it periodically expands resources. For example, when production volume increases or machine utilization rises during specific time periods, resources can be increased in advance to prevent performance degradation. Event-based scaling adjusts resources immediately when system load reaches a threshold, thereby enhancing resource utilization efficiency. For example, when there is a sudden surge in demand or when production needs to be ramped up quickly, necessary resources can be swiftly increased to maximize productivity.

Contributions and Novelty.

- *SRN-Based Modeling for Ceph S-iNAS:* We develop a comprehensive SRN model tailored to a Ceph-driven NAS environment. Unlike previous Petri net or SPN approaches that often omit dynamic scaling or address only partial system aspects, our model captures both the fundamental Ceph operations (read, write, replication) and the dynamic addition or removal of resources from the storage cluster.
- *Dual Policy Evaluation (Time-based vs. Event-based):* Our work introduces and evaluates two primary autoscaling strategies—scheduled and threshold-based. We delve deeply into how these policies respond to changing workloads, contrasting scenarios where scaling is triggered by fixed time intervals (Time-based) versus those where scaling is initiated by real-time queue/load thresholds (Event-based).
- *Extensive Sensitivity Analysis:* Beyond a baseline stationary analysis, we conduct

comprehensive sensitivity tests on key parameters such as arrival rates, client population, VM creation times, and the read/write workload composition. This investigation yields fine-grained insights into how different configurations drive throughput-latency trade-offs and resource usage.

- *Guidance for Practitioners:* By interpreting these trade-offs—particularly throughput versus latency—we offer concrete guidelines for industrial system administrators. The insights reveal how modest adjustments in thresholds or tiOme intervals can profoundly affect performance, helping practitioners make well-informed decisions on provisioning and scaling within Ceph-based NAS deployments.

Our approach illustrates how SRNs, carefully integrated with Ceph's architectural principles, can empower administrators and system architects to navigate the complexities of industrial-scale data processing. By enabling proactive resource planning and dynamic scaling decisions, the proposed framework aims to maintain high throughput, low latency, and robust fault tolerance—core objectives in real-world industrial applications.

The remainder of this work is organized as follows: Section 2 surveys related research on autoscaling, SRN modeling, and Ceph-based performance analysis, and highlights the existing gaps. Section 3 describes the S-iNAS system architecture, detailing Ceph's components and discussing how they fulfill industrial performance needs. Section 4 explains our SRN modeling approach, including the states, transitions, and reward structures that capture both write/read processes and resource scaling events. Section 5 presents our experimental setup, including default parameters and a range of sensitivity analyses exploring different arrival rates, client sizes, VM instantiation times, and job compositions. Section 6 concludes by summarizing the key findings, discussing practical guidelines, and suggesting potential extensions such as AI-driven or cost-aware scaling mechanisms.

2 Related Works

In this section, we categorize previous research related to our proposed S-iNAS framework, mirroring the layout used in Tables 1. The works span four broad themes: (i) Stochastic and Petri Net-Based Modeling, (ii) Auto-scaling or resource adaptation in cloud environments, (iii) Ceph-based or NAS-focused performance analyses, (iv) Hybrid storage and distributed system verification and (v) network and system management for distributed storage.

Below, we synthesize these threads and highlight the gaps that motivate our S-iNAS approach.

2.1 Stochastic and Petri Net-Based Modeling

Several efforts deploy Stochastic Petri Nets (SPNs), Generalized Stochastic Petri Nets (GSPNs), or related formalisms for performance or dependability in distributed systems. For example, [39] uses a GSPN to optimize resource management in distrib-

Table 1 Comparison of selected works

References	Focus	Method	Auto-scaling?	Performance focus	Gap vs. S-iNAS
[39]	GSPN for manufacturing optimization	GSPN	No	Event manufacturing control; fault tolerance	Different domain; no Ceph/NAS analysis
[40]	Private cloud performance analysis	RBD + CTMC	No	Steady-state downtime and reliability	Static design; no dynamic scaling
[41]	Performance and energy in storage	Petri Net / SPN	No	Throughput, Response time and energy usage	No auto-scaling or time/event-based strategies
[34]	Microservices auto-scaling via Petri Net	Formal Petri Net	Yes	Throughput in microservice deployments	Not focused on Ceph or storage nodes
[42]	SRN for virtual NAS evaluation	SRN	No	Availability costs	Limited scaling logic; no Ceph distribution
[43]	Adaptive data scaling for SLA	Dynamic data shard scaling	Yes	SLA improvements	Focus on DB; lacks Ceph overhead
[11]	Adaptive VM/ container scaling	Predictive model	Yes	Cost and QoS	Not Ceph-focused; lacks SRN model
[6]	Ceph design and replication	Architecture discussion	No	Scalability tests	Foundational reference; no dynamic scaling
[44]	NoSQL DB performance analysis	Experimental evaluation	No	Throughput focus	Not about Ceph or dynamic scaling
[17]	Ceph throughput benchmarks	Empirical tests	No	Manual scaling observations	Benchmarking only; no dynamic auto-scaling
[45]	Adaptive model for workload classification	Workload-based scaling policy	Yes, VM level	SLA compliance, forecast accuracy	Not Ceph-focused; lacks replication strategies
[15]	Petri Net modeling for CPS	Standard Petri Nets	No	Security logic, not throughput	No Ceph analysis or scaling
[46]	SPN/RBD for file server availability	SPN + RBD	No	Downtime analysis	Static replication; no scaling transitions
[47]	Self-adaptive scaling in microservices	Performance analysis	Yes (containers)	Microservice concurrency	Lacks Ceph overhead; no scaling strategies
[48]	Fault injection in event-based systems	Code-injection	No	Latency, fault tolerance	Different methodology; no Ceph scaling
[8]	SPN + RBD for hybrid storage	Stochastic modeling	No	Throughput and availability	No dynamic scaling; focus on local storage
[23]	Markov model for SSD GC	Markov chain	No	Performance tradeoffs	Not cluster-level; local SSD focus

Table 1 (continued)

References	Focus	Method	Auto-scaling?	Performance focus	Gap vs. S-iNAS
[9]	SPN/CTMC for HDFS	Stochastic Petri Net	No	Dependability analysis	No scaling; not Ceph
[18]	Formal verification of storage	IronFleet-like approach	No	Logical correctness	No performance or scaling analysis
Our study	Ceph based S-iNAS performance analysis	SRN	Yes	Throughput and Response Time	Ceph based data workflow(Replication etc.) uses dynamic scaling for OSD, measuring performance metrics with auto-scaling

uted manufacturing, focusing on production throughput and risk tolerance. Despite its SPN formulation, this work differs from ours by targeting manufacturing control systems rather than Ceph or network-attached storage (NAS).

Other research considers storage-related Petri Net models but omits dynamic scaling policies. For instance, [15] focuses on high-level risk/security logic for cyber-physical systems, without addressing Ceph replication overhead or time/event-based scale-outs. Likewise, [9] models HDFS reliability with SPNs/CTMC, but does not handle auto-scaling or Ceph-based distribution. Thus, while Petri Net frameworks suit distributed dependability and performance, the literature rarely combines scalable resource expansions with replication overhead in Ceph-like architectures.

2.2 Auto-Scaling Approaches in Cloud Environments

Research on auto-scaling typically addresses container-level or VM-level expansions, emphasizing either machine-learning predictions or rule-based triggers. *(i) Microservice/Container Scaling:* [34] utilizes Petri Nets for auto-scaling microservices to maintain latency/throughput SLAs. This aligns with our interest in formal modeling for scaling but focuses on container orchestration rather than block/object replication. Similarly, [47] probes container scaling strategies for concurrency, yet lacks storage-layer overhead analysis. *(ii) Predictive Scaling:* [45] applies workload classification to dynamically select forecast algorithms (LR, SVM, ARIMA). [11] also uses machine-learning predictions to scale VM/container resources. Although these reduce SLA violations, they concentrate on application or compute scaling and do not explicitly model Ceph or storage replication overhead. *(iii) Data-Shard Adaptation:* [43] proposes an adaptive controller for data shards to meet latency/throughput SLAs; however, it remains shard- or database-centric, rather than focusing on Ceph OSD scaling or SRN-based auto-scaling.

Hence, although auto-scaling is widely studied, few works integrate storage-layer expansions or Ceph OSD elasticity. Our model addresses this gap by embedding time- and event-triggered expansions directly in an SRN, capturing overhead from replication and block/object I/O concurrency.

2.3 Ceph-Based or NAS-Oriented Performance Analyses

A second body of literature explicitly covers Ceph or NAS performance but rarely unites auto-scaling with formal modeling. *Ceph Architecture and Benchmarking:* [6] is a seminal reference describing Ceph design and replication strategies. Nevertheless, it omits dynamic scale transitions in a formal framework. Similarly, [17] conducts empirical Ceph benchmarking under cluster expansions but lacks a unifying SRN model. *NAS Virtualization with SRNs:* [42] employs SRNs to assess availability and downtime for a virtualized NAS. Though instructive, it does not incorporate time/event-based expansions or Ceph distribution overhead. By contrast, our S-iNAS introduces SRN-based auto-scaling transitions (both vertical and horizontal). *Cloud Storage in Private or NoSQL Contexts:* [40] looks into private-cloud storage availability using RBD+CTMC, while [44] experimentally measures throughput/latency for NoSQL-based solutions. Neither addresses Ceph's block/object replication nor dynamic scaling triggers. Thus, prior Ceph or NAS works tend to focus on static or partially manual expansions. By contrast, our approach systematically encodes time/event-based scaling of Ceph resources in an SRN, enabling thorough performance-latency trade-off analysis.

2.4 Hybrid Storage and Distributed System Verification

Some complementary studies delve into specialized storage designs or verification frameworks. *Hybrid HDD-SSD Models:* [8, 41] adopt Petri Net or RBD methods to estimate performance/energy usage or reliability in mixed HDD-SSD environments. While these confirm the efficacy of net-based approaches for local storage, none incorporate large-scale Ceph expansions, time/event triggers, or replication overhead. *Distributed Event and Verification:* [48] uses code-injection for event-based system fault injection, lacking, however, an SRN model or auto-scaling logic. In parallel, [18] employs a formal correctness methodology (inspired by IronFleet) for storage, focusing on logical proofs rather than performance-latency analyses under dynamic scaling. *SSD GC Overheads (Markov):* [23] concentrates on SSD garbage collection overhead and wear-leveling via mean-field Markov modeling, omitting cluster-level replication or scale-out/in. Though instructive, these studies underscore partial aspects—fault injection, formal verification, or local SSD optimizations—without addressing distributed Ceph expansions in response to shifting I/O demands.

2.5 Network and System Management for Distributed Storage

Industrial data platforms need storage that sustains steady throughput and low latency during bursts and diverse loads. Ceph clusters provide replication and capacity expansion, though internal decision logic imposes performability trade offs. Studies indicate autoscaling with inteligent monitoring for cloud storage, whereas surveys classify it mainly for compute VMs and containers, neglecting storage. Jennings and Stadler [21] propose a Monitor, Analyze, Plan, Execute (MAPE) control loop architecture that optimises performance and service level agreements. Lorido Botran [35]

review threshold based and machine learning predictive autoscaling that preservs elasticity, focusing on load balancing and CPU or RAM instead of storage.

Elastic auto scaling in service and network management: PRESS first predicted workload intensity to trigger VM allocation [7]. Reinforcement learning soon entered resource controllers for multi tenant clouds [49]. NFV studies followed, HyScaler joining vertical with horizontal moves [10]. Analytical optimisation shaped cost aware choice of burstable instances [50]. However, storage elasticity remains thin. AdaptScale resizes shards on the fly to meet latency and througput aims [43]. This mechanism covers only NoSQL and omits Ceph redistribution cost. Ceph Nautilus (2019) retunes placement groups for balance [51]; it neither adds nor drops OSDs under load. Extra capacity and replication impacts are seldom modelled. Gudu's manual OSD expansion [17] lifts throughput, tho rebalancing cost and replication delay still occurr. No work had merged real time autosacling with a formal Ceph model until S-iNAS. Its time based and event based OSD policies echo earlier elasticity ideas [7, 10] while pushing them into the storage tier. The SRN driven engine complements rule based and reinforcement schemes, delivering a performability view on when and how Ceph can grow, offering guidance for storage heavy industrial workloads where tight QoS targets and replication overheads coexist.

Monitoring and resource management: Monitoring and resource stewardship anchor S-iNAS and close the control loop. The platform contiuously captures I/O rates, latency traces and OSD states, then invokes scaling, echoing rich telemetry practice in network operations [52]. Multidimensional monitors such as the three dimensional cloud model of Marquezan et al. [53] motivate S-iNAS to stream live metrics into policy logic. Similar to in band telemetry for packet visibility [20, 22], embedded probes reveal storage health, complete the MAPE K loop (MAPE + Knowledge), and feed the SRN that executes analyze and plan phases, steering self scaling and optimisation. Researchers gather counters like IOPS, latency, bandwidth and OSD utilisation, then flag overload or SLA breaches. Ceph supplies Monitor and Manager daemons that log OSD health, link delay and cluster status, exploitation for optimisation remains sparse. Recent analysises apply machine learning on logs to pinpoint hotspots or failure, then tweak configuration early, while centralised controllers akin to software defined networking gain attention. Li and Wang [24] present a cloud storage frame where an SDN controller harvests bandwidth and node load contiuously, reallocates data, considers network factors and hardware diversity such as HDD versus SSD and OSD CPU strength, thereby yielding superior load balance plus higher read write throughput versus Ceph's CRUSH algorithm. This shift toward unified oversight of compute, storage and network under one control plane enhances system performance.

Storage resource allocation and performance assurance: System management research examines data placement and scheduling to keep storage fast and available. Torres [40] merge simulation with hierarchical analysis of replicated private clouds, showing replica layout and hot spare pools govern recovery and response. Silva and Callou [46] presents a reliability model for file server clusters, it quantifies how node churn or tuned backups change availability, while autoscaling is almost untouched by thier analysis. Li [24] model large SSD pools with Markov chains, balancing throughput against drive life and supporting the earlier SDN scheme. The

evidence suggests allocation, replication, and scheduling need coordination; extra devices rarely solve performance.

Fault management and reliability orchestration: link directly to S-iNAS replication logic. Ceph's data copies across OSDs express fault tolerance, a topic in network and service management. S-iNAS quantifies replication overhead and recovry expense in its SRN, framing the trade off between redundancy and throughput. Fault work covers detection, alarm correlation, and repair. By preemptively tuning the OSD pool to track load, S-iNAS adds a preventive scheme against overload. The brief slowdown during scale out matches cases where extra links or standby nodes raise resilience and add latency or consistency cost. Past dependable storage and network studies analysze the need to measure this burden. Via cost analysis, S-iNAS supports resilience modelling, showing how scale thresholds or timing change performance, durability, and recovery time goals. The method complements machine learning driven fault management in NFV, supplying a transparent white box, model driven route to reliability.

Resource orchestration: Within network operations, resource orchestration denotes intentional cooridination of recources and services, covering NFV MANO engines that place VNFs and cloud controllers spreading containers across clusters. S-iNAS automates Ceph, instantiates OSD machines, adds them to the cluster, and calls CRUSH to rebalance data, forming a storage specifc workflow. Recent taxonomies already reserve an orchestration section, confirming its status in service management for virtualised and software defined infrastructures. By coupling performance policies with storage orchestration, S-iNAS mirrors Kubernetes and OpenStack that coordinate resources at scale, whereas its analytic model guides every action. Thus scaling decisions become proactive and cost aware, offering insight to intent driven cloud controllers. The formal policy inside S-iNAS acts as an intent, for example sustain needed througput with minimal expense, and the orchestrator realises this aim via adaptive resource scaling.

Intent-based management and AI-driven policies S-iNAS aligns with intent based network managment and AI driven policy, which seek to move beyond static expert rules. An operator declares a latency or throughput target, and a closed loop controller reallocates OSDs, bandwidth, and other recources until the target holds. The SRN model plus sensitivity analysis inside S-iNAS already provide the quantitative base an intent engine needs to time scaling decisions. Paired with an AI driven controller, S-iNAS could learn optimal scaling or retune thresholds from workload forecasts, following recent Fault, Configuration, Accounting, Performance, Security (FCAPS) studies that apply machine learning across fault, configuration, accounting, performance, and security tasks. Those studies report better fault prediction, tuning, and resource allocation. Because the S-iNAS analitical model is mathematical and explainable, operators see why each action is taken, supporting trust in closed loop control. Linking S-iNAS to an AI policy engine also serves the zero touch Network and Service Management vision, where monitoring, analysis, and actuation run autonomously.

Storage centric management and Formal performance modelling: In band Network Telemetry (INT) yields fine grained obervsation of congestion and replication delay, closing the loop [20]; integer programming guides VNF on network and stor-

age [54]. Ceph scaling experimental; Libra provides Kubernetes pod scaling for storage gateways [55], Nguyen et?al. proposed vector rate load balancing for user plane gateways with OSD counts [56], and S-iNAS supplies an analytc backbone for those prototyps. Queueing theory dominates elasticity studies, whereas SRN extensions address stochastic loads, and Jennings and Stadler outlined a MAPE encouraging SRN adoption [21]. Torquato et?al. modelled VM migration rejuvenation with SRNs, showing reward structures capture bursty traffic [57]; this work embeds Ceph replication and scaling tokens inside an SRN framework.

2.6 Positioning of Our S-iNAS Approach

Existing literature on Petri Net or SPN modeling, as well as autoscaling and Ceph-based performance, remains fragmented, lacking a unified SRN-based approach that simultaneously addresses both dynamic expansions (vertical/horizontal) and replication overhead. Traditional Ceph performance research typically examines static clusters or partially manual expansions. Meanwhile, auto-scaling efforts often concern application-layer microservices or compute resources, rather than focusing on storage nodes or OSD daemons.

Our proposed S-iNAS framework closes this gap by:

- Embedding Time-based and Event-based expansions (Scale Up/Down, Scale In/Out) within a unified SRN model.
- Handling Write and Read operations under Ceph-like block or object replication logic.
- Enabling thorough Sensitivity Analyses for arrival rates, client concurrency, VM instantiation overhead, and write-read ratios.
- Demonstrating how to interpret the throughput vs. response time trade-offs for different auto-scaling triggers.

Our work extends network management knowledge in three dimensions. First, it unifies Ceph read, write and replication paths with auto scaling triggers inside one SRN, while earlier SRN studies address compute layers only [57]. Second, it contrasts time driven schedules with threshold triggers across arrival rates, client concurrency and VM bootstrap delay, an evaluation absent from compute centric surveys [7, 50]. Third, its sensitivity analysis exposes non-monotonic latency surfaces where aggressive thresholds introduce oscillation, refining prior observations on tandem queue scaling [58]. Thus, we provide a more holistic lens for analyzing and optimizing Ceph-based S-iNAS deployments, ensuring robust, dynamically scalable storage services even under variable workload intensities and large data volumes. The S-iNAS study enriches management literature by providing a rigorous performance centric model for Ceph elasticity. Integrating its SRN methodology with adaptive control strategies and holistic objectives appears to be a productive path for forthcoming investigations.

The S-iNAS study establishes a platform that invites several follow on investigations in network and systems management. A first task is to *extend the SRN model to represent the system more faithfully*; adding *OSD capacity limits and heterogeneous*

workload models lets analysts quantify how memory saturation, file size distributions, and random or sequential access patterns influence Ceph performance[37], using extra capacity places or an SRN with an explicit I/O queue. A second avenue is to *incorporate cost and energy efficiency metrics* in autoscaling assessment. S-iNAS evaluates throughput and latency, whereas operators also face virtual machine expense and cluster power draw, and recent research links performance with cost in cloud autoscaling or employs SRN for reliability and cost estimation in SDN infrastructure. Hence S-iNAS needs a *multi objective* scheme that balances performance, latency, reliability, and cost for real service governance. Third, work on *applying AI and reinforcement learning to autoscaling decisions* gains momentum. Instead of static thresholds, an RL agent derives a near optimal policy from simulation; Jiménez et?al.?(2023) reported superiority over fixed threshold schemes in dense multi tenant settings, jointly optimising latency and replica count. Insights from S-iNAS, such as the twenty five percent thresold that yields top performance, can seed learning algoritms, and the SRN itself serves as a transparent simulator that accelerates training. In parallel, short term load forecasts with LSTM and Bayesian optimisation can *automatically adjust autoscaling parameters to current system conditions*. The surge towards *intent-based networking and zero touch automation* also aligns with S-iNAS. IBN engines in 5 G and 6 G declare intents like "keep storage service latency below one second" and rely on models to choose actions. Because the SRN foresees system response, a management loop queries whether an event threshold of thirty percent satisfies the latency intent and proceeds; a *combination of formal modelling and AI control* is therefore likely, with SRN enforcing constraints and AI refining operation. Deployment realities remain. The *practical deployment* of S-iNAS on Kubernetes or OpenStack needs a module that orchestrates OSDs, guided by current S-iNAS advice on scaling period and queue thresholds; engineers must handle Ceph Monitor integration, startup delay of new OSDs, and data migration or replication when shrinking the cluster, drawing on experience from web autoscaling and VNF management mechanisms discussed at recent CNSM, NOMS, and IM forums. Industrial storage and network service management is moving toward *autonomous, intent-driven, AI-supported solutions*, where formal SRN modelling delivers correctness and quantitative insight, while machine learning delivers agile optimisation; S-iNAS has laid essential groundwork by proving the merit of SRN based performability analysis and a detailed comparison of two autoscaling strategies, establishing a firm foundation for subsequent research in the Network and Systems Management community.

3 System Architecture

3.1 Overview of Ceph and Its Industrial Significance

This section provides a concise overview of Ceph's fundamental concepts and architecture. Following this, we will delve into the industrial significance of Ceph and its practical applications across diverse sectors. According to [6], Ceph is an open-source, Software Defined Storage platform capable of concurrently handling Block, File, and Object Storage. Its architecture aligns with cloud computing and software-

defined principles, ensuring scalability and flexibility. Due to its resilient design, Ceph is widely used in industrial settings for storing and analyzing large data streams generated by distributed sensors and actuators. In such industrial scenarios, real-time data analytics is vital for decision-making, accident prevention, and operational efficiency. For instance, historical accident data can be examined to discover root causes and mitigate similar future occurrences. By integrating and processing large volumes of data, Ceph bolsters system stability and increases operational productivity. Its capacity to deliver multiple storage backends—Block, File, and Object—attests to its flexibility in meeting diverse requirements, thereby establishing it as a key storage technology for data-centric operations.

3.2 Ceph Storage Backends

`Ceph Block Device (RADOS Block Device, RBD)`: This high-performance, block-based storage solution is particularly suited for virtualization platforms and databases. It integrates seamlessly with widely used virtualization environments, such as KVM, Xen, OpenStack, and CloudStack, enabling straightforward allocation and management of block storage for virtual machines. Employing snapshots and replication for data protection and disaster recovery, RBD delivers high availability and strong scalability.

`Ceph Object Device (RADOS)`: This object-based storage solution is especially advantageous for web applications and large-scale data storage. By offering direct data access through the RADOS Gateway API, it eliminates the need for intermediary operating system layers, thereby granting users greater flexibility in data manipulation and management. Each object includes data, metadata, and a unique identifier, which simplifies data retrieval and administrative operations.

`Ceph File System (CephFS)`: A POSIX-compliant distributed file system that operates via Filesystem in User Space (FUSE) for comprehensive compatibility. It can work alongside NFS, SMB, or other file-sharing protocols while also supporting the mounting of object storage. By decoupling metadata operations from actual data storage, CephFS mitigates overhead and bolsters system stability. At least one Metadata Server (MDS) must be present in the cluster for metadata transactions and optimized throughput, underscoring the critical importance of metadata in file-based workloads.

Overall, Ceph delivers an effective solution for data-intensive and latency-sensitive applications by incorporating block, file, and object storage functionalities. Its strong focus on reliability and fault tolerance makes it particularly appealing for industrial scenarios where continuous sensor- and actuator-generated data necessitates uninterrupted stability and operational effectiveness. By providing specialized storage modes, Ceph ensures that distinct operational requirements—ranging from high-bandwidth transfer to high-availability replication—are fulfilled.

3.3 Ceph Cluster Components

A Ceph cluster comprises a Monitor, Manager, OSD, and Pool, collectively maintaining system availability and scalability. Optionally, a Metadata Server (MDS)

can be included to support file-based operations under CephFS. `Ceph Monitor:` This component continuously monitors and maintains the cluster's state. To preserve quorum and ensure high availability, it is typically deployed in clusters of three or five monitors. The monitor also manages and updates the cluster map, thus guaranteeing data reliability by transmitting cluster status and control information to connected clients. `Ceph Manager:` Working in conjunction with the monitor, the manager gathers performance indicators and cluster status information, providing real-time insights into system health for administrators. Its native interfaces may also be used with third-party monitoring solutions. `Ceph OSD (Object Storage Daemon):` As the primary data store, each OSD normally corresponds to a storage device, processing client read and write requests directly. Additionally, replication tasks occur in the OSD layer, ensuring data integrity in the event of device or network failures. `Pool:` Acts as a logical intermediate layer between Ceph clients and physical OSDs. Clients store RADOS objects within designated pools, and the CRUSH algorithm automatically distributes these objects among the correct OSDs. This design achieves notable scalability and high availability by isolating logical object placement from the underlying hardware.

3.4 S-iNAS in Industrial Domains

Figure 1 depicts various contexts wherein Scalability in Industrial Network Attached Storage Systems (S-iNAS) employs Ceph in industrial domains. For example, conveyor systems, a fundamental element in automated warehousing and manufacturing, require the effective capture, storage, and analysis of data to enable real-time decisions and predictive maintenance. With its capability for large-scale data ingestion from myriad sensors and actuators, S-iNAS facilitates more efficient operations and data-informed resource allocation on production lines.

S-iNAS underscores the significance of conveyor systems in warehouses and manufacturing processes. By aggregating and analyzing real-time sensor and actuator data, S-iNAS enables timely detection of anomalies, mitigates downtime, and consequently boosts total throughput. The interplay between automated material handling and data-centric strategies remains fundamental to modern smart factories. Crucial maintenance factors for industrial conveyor systems include the following. *Debris Management:* A range of materials can deposit foreign objects, thereby

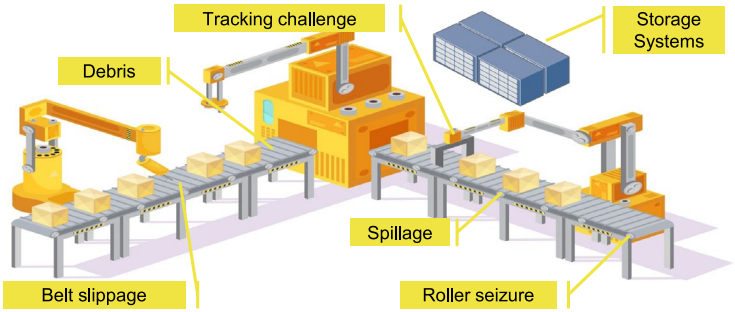


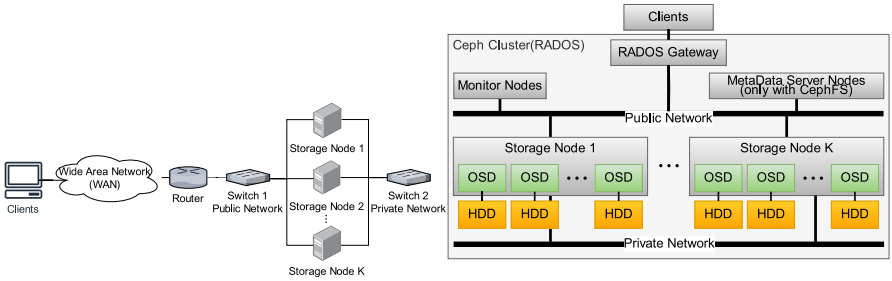
Fig. 1 S-iNAS illustration in industry [59]

increasing inefficiencies and wear. Real-time, sensor-based detection aids in the proactive removal of debris. *Belt Slippage:* Variances in tension, overload, or belt wear can undermine accurate material handling. Continuous monitoring and belt-tension adjustment allow S-iNAS to avert slippage and avoid throughput reduction. *Spillage:* Unintended material discharge can compromise safety and inflate operational costs. Sensor feedback enables preventative measures and route optimization in a timely manner. *Roller Seizure:* Blocked or stalled rollers can precipitate sudden system downtime. Temperature and rotational speed data, combined with predictive analytics, assist in preempting these incidents. *Tracking Challenge:* Conveyor path deviations may result from misalignments or external disruptions. Through real-time monitoring and corrective action, the system confines these deviations and sustains uninterrupted throughput. Gathering, storing, and analyzing sensor and actuator data are thus vital to preserving conveyor uptime and efficiency. By collecting this high-volume industrial data within an S-iNAS architecture, routine system checks benefit from advanced analytics, leading to superior reliability and more granular process oversight.

3.5 Network Topology and System Architecture in S-iNAS

As shown in Fig. 2a, the S-iNAS network structure consists of geographically distributed clients connecting to storage nodes via a Wide Area Network (WAN), ensuring dependable access and storage capacity. Each storage node offers Ceph's block, file, or object services, tailored to meet different performance and capacity demands.

In this configuration, a Public Network (Switch 1) manages data exchanges with external clients over the internet. Meanwhile, a dedicated Private Network (Switch 2) oversees critical internal operations such as data placement, replication, metadata management, and system monitoring. `Data Placement:` Employs the CRUSH algorithm to balance data distribution across the cluster, mitigating performance hotspots and evening out node utilization. `Data Replication:` Ensures high availability by duplicating objects to multiple OSDs, thereby preventing data loss should hardware issues arise. `Metadata Management:` Involves specialized servers (MDS) that maintain a separate metadata layer to expedite quick lookups and preserve system responsiveness for file-based requests. `Monitoring & Health`



a: Network Architecture b: System Architecture

Fig. 2 Network and system architecture of S-iNAS

`Checking`: Continuously tracks OSD statuses, network latencies, and overall cluster integrity. Prompt anomaly detection facilitates swift administrative intervention, minimizing the likelihood of data loss and service disruptions.

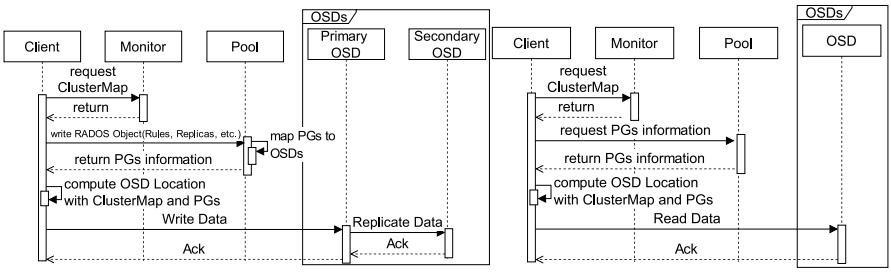
Collectively, the interplay among these components underpins a system with fault tolerance, substantial throughput, and low latency. The architecture ensures reliable data accessibility across dynamically shifting workloads typical of extensive industrial or enterprise deployments. By integrating resource management, replication, and robust monitoring features, S-iNAS represents a flexible yet dependable storage ecosystem suited to contemporary data-intensive and mission-critical operations.

3.6 Data Operations in S-iNAS

Ceph is an exceptionally robust and versatile distributed storage platform, meticulously designed to deliver superior performance, reliability, and scalability. Based on the conceptual framework presented in Fig. 2b, a Ceph-driven, scalable Network Attached Storage (NAS) system can be envisioned as a cluster of physical or virtual machines—hereafter referred to as nodes—that collaborate to form a unified environment. Each node fulfills specific functions, such as cluster monitoring (Monitor Node), metadata administration (Metadata Server Node), performance oversight (Manager Node), and data processing (OSD Node).

In this architecture, seamlessly adding new nodes or Object Storage Daemons (OSDs) increases overall storage capacity without incurring downtime. Ceph's internal distribution and load-balancing mechanisms automatically reallocate and rebalance data across newly introduced resources, thereby creating a fault-tolerant, high-performance storage solution well-suited for applications requiring large data capacities and high-speed I/O throughput.

Figure 3 is a Data Operation Sequence Diagram of the Write Operation and Read Operation in S-iNAS. A common necessary step for both operations is finding the location of the OSD where data will be stored or read. Figure 3a describes the Write Operation: the client first requests the ClusterMap from the monitor. When sending a write request for the Pool's PG using a RADOS command with parameters such as Rules and the replication count, the PG information is returned. The client then calculates the OSD location using the ClusterMAP and the PG. After directly accessing the Primary OSD to perform the write, the data is replicated to Secondary OSDs



a: Write Operation b: Read Operation

Fig. 3 Data operations sequence diagram in S-iNAS

located in different domains according to the replication factor (assumed to be 2 in this work), after which the Data Operation is completed. Figure 3b illustrates the Read Operation: to find the OSD location, the client requests the ClusterMap from the monitor. Subsequently, after requesting and receiving the PG from the Pool, the client uses the ClusterMap and the PG to calculate the OSD location. The client then directly accesses the calculated OSD location to read the data. In this study, the processes depicted in Fig. 3 are modeled using SRN and incorporated into the Monitor Node and SNode i...j components of Fig. 4. More detailed explanations are provided in Sect. 4.

3.6.1 Node and Policy Definitions

`Node Type Policy`: Within a Ceph cluster, each node is dedicated to a particular role:

`Storage Node`: Such a node includes one or more Object Storage Daemons (OSDs). Each OSD is a software component bound to a specific storage device—whether physical or logical—thereby handling read and write requests. For instance, a single storage device can be partitioned into multiple logical volumes, each hosting a separate OSD. In larger setups equipped with multiple disks or SSDs, administrators may install additional OSDs to expand storage capacity. By distributing and replicating data across these OSDs, Storage Nodes assure both availability and resilience in the event of hardware failure.

`Monitor Node`: Continuously tracks the cluster's state and regulates data distribution through the CRUSH map and Placement Groups (PGs). PGs function as logical subdivisions used to group and replicate data objects, whereas the CRUSH

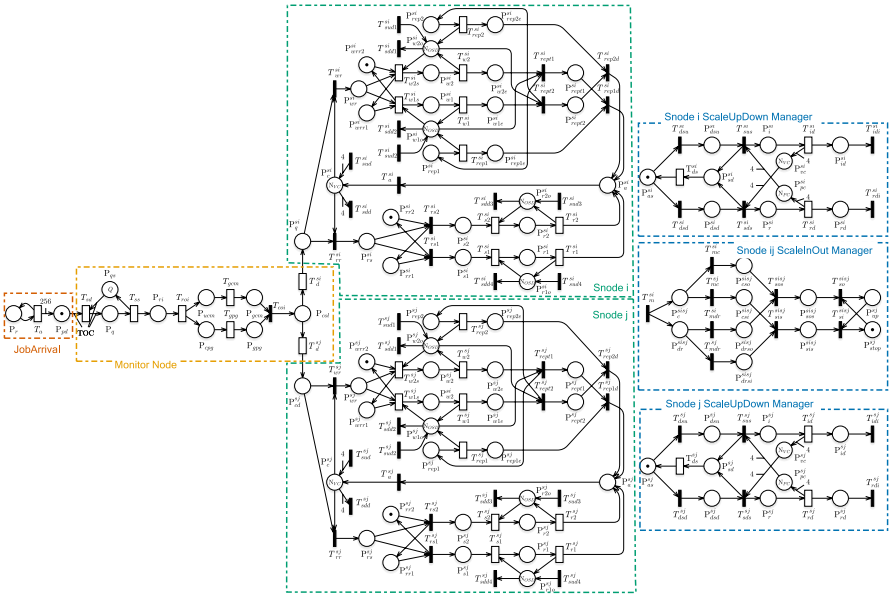


Fig. 4 SRN system model

map designates the precise placement of these groups for optimal load balancing. A typical Ceph installation employs a quorum of three or five Monitor Nodes to maintain cluster stability under node failures. At least one Monitor Node is indispensable for preserving the CRUSH map and the overarching cluster status.

`Manager Node`: Working in concert with the Monitor Node, the Manager Node continuously monitors key performance metrics, resource usage, and other critical indicators across the cluster. This component proactively adjusts cluster resources to preserve optimal service levels. Notably, one node can combine multiple roles—for instance, a node may simultaneously host an OSD and a Monitor—thereby enhancing resource efficiency.

`Data Type Policy`: At the microservice level, system traffic is categorized into two operations: Write and Read, each initially assigned a 50% probability. While real-world workloads may exhibit different mixes—databases, for example, may generate disproportionately high write traffic—this model presupposes an even distribution of read and write requests.

`Replication Policy`: This defines the number of data copies retained for high availability. Common replication factors include 0 (no replication), 1 (no redundancy beyond the primary copy), or 2 (one additional replica). Although write operations require replication to safeguard against data loss, read operations do not. In the present study, we assume that one secondary replica is maintained alongside the primary copy, without detailing the specific data placement. Replication adheres to a Peer-to-Peer model[19].

`Scaling Policy`: Auto Scaling dynamically modifies cluster resources—either adding or removing OSDs or entire nodes—in response to workload fluctuations. Both vertical and horizontal scaling are examined:

- *Scale Up:* Improves resources within an existing node—for instance, by adding OSD daemons or using faster hardware—thus augmenting capacity to accommodate heavier I/O demands.
- *Scale Down:* Relinquishes resources within an existing node, reducing operational overhead by deactivating surplus OSDs.
- *Scale In:* Lowers the total number of active nodes, minimizing infrastructure expenses when workloads drop below a predetermined threshold.
- *Scale Out:* Expands the cluster by incorporating additional nodes, thereby increasing overall storage capacity and computational power to handle sustained or escalating workloads.

In the present work, we restrict our modeling and evaluation to a Ceph cluster comprising Monitor, Manager, OSD, and Pool entities, with the object storage component. We explore essential Ceph operational procedures—ranging from data distribution (CRUSH mapping) and replication to job processing—to highlight key performance considerations for designing and enhancing Ceph-based storage frameworks.

4 Stochastic Models

Our proposed performance evaluation approach adopts a SRN model [37, 60], wherein tokens represent distinct states and transitions in a scalable NAS environment. In this framework, *places* (depicted as circles) retain tokens that reflect system states, while *transitions* (depicted as bars) govern the flow of tokens based on probabilistic and timing distributions. For instance, P_n^{si} denotes a specific state *n* at storage node *i*. By carefully defining places, transitions, and arc connections, the SRN can capture various modes of operation, from normal processing to auto-scaling events and resource releases.

4.1 SRN Components and Their Interplay

As illustrated in Fig. 4, which operates according to the data operation flow of Fig. 3 in the system architecture of Fig. 2, our SRN approach integrates four main components:

- Job Arrival: Inbound read/write requests that arrive via RESTful HTTP APIs to the Ceph cluster.
- Monitor Node: Responsible for generating and maintaining the CRUSH map, computing Placement Groups (PGs), and identifying the appropriate OSD locations for incoming jobs. We refer to this set of steps as the OSD Discovery Workflow.
- $\underline{\text{Snode}\, i, j:}$ These are storage nodes that handle both read and write operations. They also adjust resource allocations dynamically in response to changing workload demands.
- ScaleUpDown/InOut Manager: This entity oversees threshold-based scaling decisions, which may be either time-based or event-based. Depending on the system's current state, it can scale up or down the number of OSDs (vertical scaling) or scale in or out the number of nodes (horizontal scaling).

By covering both the job lifecycle (arrival, OSD discovery, I/O processing) and the auto-scaling logic (time-based or event-based), the SRN captures the core behaviors of the S-iNAS environment.

4.2 Token Flow and System States

If a particular storage node (Snode *i*, *j*) detects higher-than-anticipated load or surplus idle resources, it triggers a scaling event. During such events, individual OSDs or even entire nodes can be added or removed from the system to maintain optimal performance. Throughout this procedure, tokens traverse various operational, maintenance, and replacement states, following probabilistic transitions that mimic the stochastic nature of real-world NAS deployments.

By meticulously tracking these token movements, system architects and administrators can gain critical insights into resource allocation strategies, replication methods, and maintenance protocols. Through the SRN model, one can pinpoint the

interrelationship between throughput, latency, and available resources, ultimately guiding more informed decisions about provisioning and scaling in large-scale Ceph ecosystems.

In essence, the SRN-based model highlights the fundamental roles of the Monitor Node (for OSD discovery and PG computation), the handling of read/write requests across multiple storage nodes, and the direct impact of scaling policies on service quality. Consequently, this model provides a practical framework for optimizing data storage operations within Ceph deployments, ensuring that large-scale systems operate at peak efficiency and resilience.

4.3 Job Arrival and Monitor Node

Interaction between Job Arrival and Monitor Node. As shown in Fig. 5, the S-iNAS (Smart Intelligent Network Attached Storage) system model manages Read/Write Operations concurrently, while the Monitor Node governs the OSD Discovery Workflow. For example, a client using web-based services or RESTful API calls can initiate data storage or retrieval. In such scenarios, the Ceph Client spawns multiple threads to interact with the Ceph Cluster asynchronously. The variable TOC (Thread of Client), typically set to 16, determines how many concurrent threads operate [17].

Jobs arrive according to a Poisson process with mean inter-arrival time $1/\lambda$. Each incoming job is assumed to handle a 1 GB data object, transferred or accessed in 4 MB segments. Under these assumptions, the Timed Transition T_a generates Job Requests and places them into P_{pd} through an arc that creates 256 tokens (each token symbolizes a 4 MB data block). The total thread count is governed by TOC, while T_{sd} corresponds to P_{qs} (the system's Queue Capacity). Hence, Job Requests remain in P_q until transitioning to T_{ss} (the StartService) with probability $1/\epsilon$. Jobs that require OSD information stacked in P_{ri} (Require the OSD Information) are fired according to the probability T_{roi} (Request the OSD Information).

Afterwards, each Job updates Ceph's CRUSH MAP, determines the relevant Placement Group (PG), and retrieves OSD location data via T_{roi} (request OSD information). Upon completion, the CRUSH MAP is updated, and the job proceeds to T_{gcm} (GetCRUSHMAP) for PG computation. It then moves on to T_{gps} (getPGs) for

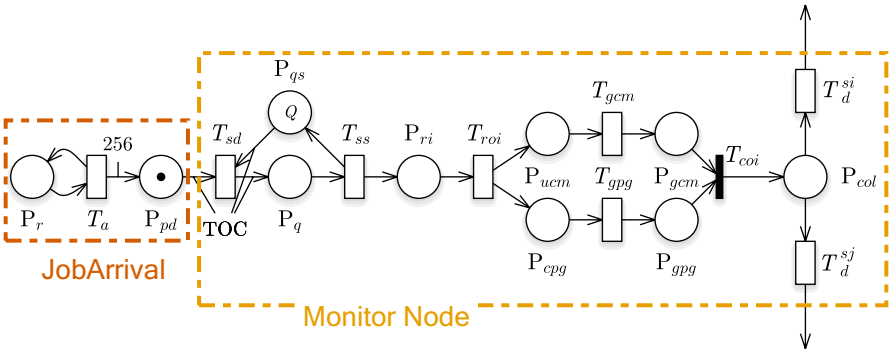


Fig. 5 Job arrival and monitor node model

OSD-specific location data collection. Finally, once these steps conclude, T_{coi} (ComputedOSDLocation) fires. After updating the CRUSH MAP and calculating the Placement Group, the OSD location information can be immediately known, allowing for direct access to the OSD.

4.4 Modeling Storage Nodes: Snode $i \ldots j$

Figure 6 illustrates how multiple Storage Nodes ($i \ldots j$) operate. When the Monitor Node amends the CRUSH MAP for a specific job and identifies the relevant OSD, P_q^{si} in Snode i acquires a token holding that OSD-related information. The variable N_{VC} denotes the number of currently available OSD resources in the node.

If both P_q^{si} (the job queue within Snode i) and P_c^{si} (the node capacity) have tokens, a job is allocated as either a Write Operation or a Read Operation (50% probability each), populating P_{wr}^{si} (write queue) or P_{rr}^{si} (read queue), respectively.

Write Workflow. A token in P_{wr}^{si} (Write Request) indicates a write request. If P_{w1o}^{si} (write to 1 OSD) or P_{w2o}^{si} (write to 2 OSD) is idle, a RoundRobin process (controlled by P_{wrr2}^{si} and P_{wrr1}^{si}) assigns the job. If P_{wrr2}^{si} holds a token, T_{w1s}^{si} (write 1 start) fires to initiate the first Write Operation.

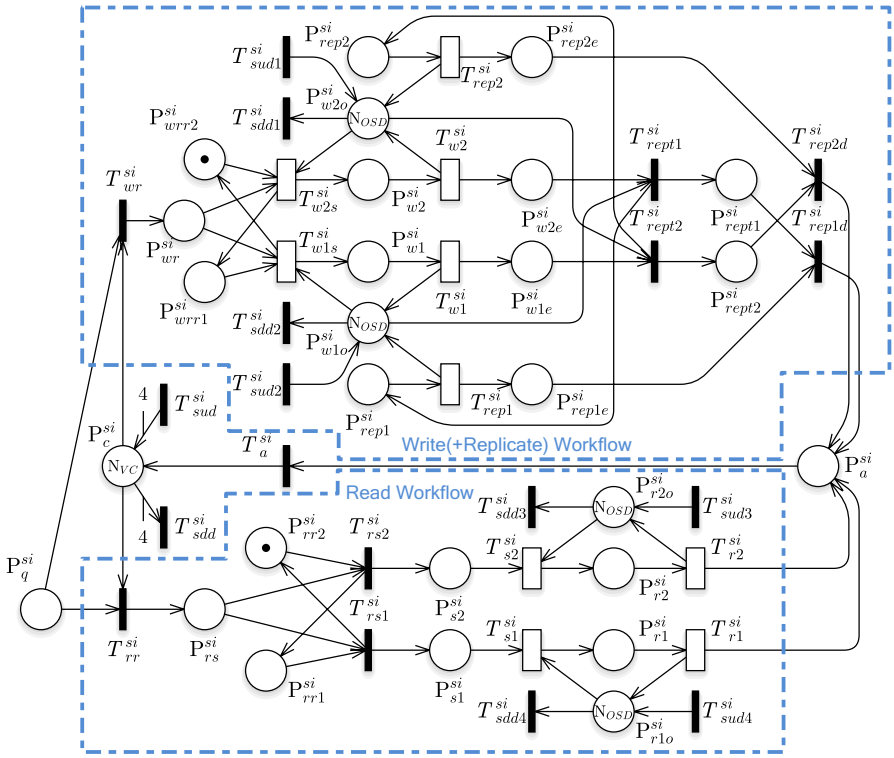


Fig. 6 Write(+replicate) and read workflow of snode i model

Once T^{si}_{w1s} fires, if P^{si}_{w1} (write 1) is free, it handles the write with an exponential service time of mean $1/\lambda_{write}$. Upon finishing, the token moves to P^{si}_{w1e} (write 1 end), denoting that replication is ready to begin.

If P^{si}_{rep2} (replicate to 2) is empty, T^{si}_{rept2} triggers replication with mean $1/\lambda_{replicate}$. When replication completes, the T^{si}_{rep2e} immediate transitions the token to P^{si}_{a} (ack), signaling a successful write. Finally, the Snode OSD resource is freed and returned to P^{si}_{c}.

Read Workflow. For read operations, P^{si}_{rs} holds a token. On arrival of a new read job, RoundRobin logic chooses T^{si}_{s2} (read 2) or T^{si}_{s1} (read 1). If P^{si}_{r1o} (read 1 OSD) is free, P^{si}_{r1} (reading 1) serves the read request with an exponential mean of $1/\lambda_{read}$. Upon completion, the T^{si}_{r1} transitions, returning the OSD resource to P^{si}_{c} and finalizing the read procedure.

The immediate transitions for OSD(T^{si}_{sud} (ScaleUpDone), $T^{si}_{sud1..4}$(ScaleUpDone1..4), T^{si}_{sdd} (ScaleUpDone), $T^{si}_{sdd1..4}$(ScaleDownDone1..4)) explan with the following Subsect. 4.5.

4.5 Auto Scaling Manager

Figures 7a and b depict the resource management logic within Snodes $(i \ldots j)$. Specifically:

Scale Up/Down. In Fig. 7a, T^{si}_{dsu} (detect scale up) fires if the required conditions are satisfied, signaling that P^{si}_{dsu} is active. The system consumes tokens from P^{si}_{pc} (physical capacity) via T^{si}_{dsu}, then activates T^{si}_{sus} (scale up strategy). As a result, P^{si}_{i} (instancing) and P^{si}_{sd} (Scaling Delay) commence. The transition T^{si}_{ds} (Delay Scaling), governed by $1/\lambda_{DelayScaling}$ for time-based scaling, prevents overly frequent scale toggles. Additionally, P^{si}_{as} (Await Scaling) must hold tokens before further scaling steps proceed.

After T^{si}_{id} (instancingDone) fires, the OSDs creation completes, transferring tokens to P^{si}_{vc} (virtual capacity) and P^{si}_{id} (InstancingDone). and then the immediate transitions($T^{si}_{sud1..4}$(ScaleUpDone1..4)) fires to increase the number of OSDs in Snode i. This effectively enlarges Snode i's capacity. Then T^{si}_{sud} (ScaleUpDone) adjusts P^{si}_{c} (capacity), and T^{si}_{idi} finishes the Scale Up process.

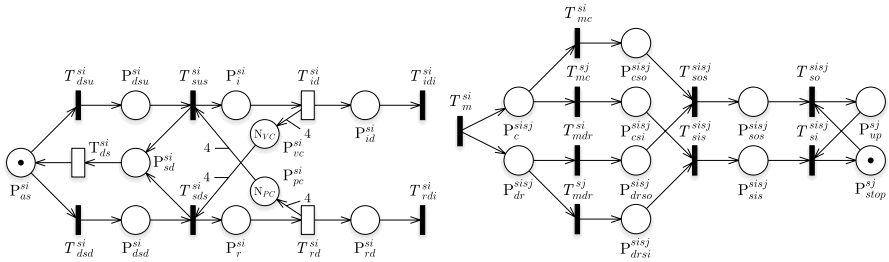


Fig. 7 Auto scaling manager in S-iNAS

For Scale Down, T^{si}_{dsd} (detect scale down) triggers if P^{si}_{dsd} is active, drawing tokens from P^{si}_{vc} and moving them to P^{si}_{r} (releasing) via T^{si}_{sds} (scale down strategy). Following resource release by Snode *i*, T^{si}_{rd} fires, placing tokens into P^{si}_{rd}. and then and then the immediate transitions($T^{si}_{sdd1..4}$(ScaleDownDone1..4)) fires to decrease the number of OSDs in Snode *i*. Subsequently, T^{si}_{sdd} (ScaleDownDone) decrements N_{VC}, and T^{si}_{rdi} completes Scale Down.

Scale In/Out. Activated by T^{si}_{m} (monitoring), tokens in P^{sisj}_{c} (get sisj Capacity) and P^{sisj}_{dr} (get sisj DataRate) gather capacity and input data rate for Snode *i* and *j*. If these measures surpass those of Snode *j*, T^{si}_{mc} (Capacity i >j) and T^{si}_{dr} (Datarate i >j) (fire, depositing tokens in P^{sisj}_{cso} (Capacity Scale Out) and P^{sisj}_{drso} (DataRate Scale Out). Once in these places, the Scale Out Strategy triggers transitions to P^{sisj}_{sos} (Scale Out Strategy). Simultaneously, tokens in P^{sj}_{stop} move to P^{sj}_{up}, allowing the newly added Snode *j* to accommodate subsequent requests.

Hence, the Auto Scaling Manager orchestrates resource distribution for the Snode in real time, incorporating both vertical (Scale Up/Down) and horizontal (Scale In/Out) adjustments. This ensures that Ceph clusters dynamically respond to workload variations with improved resource efficiency, reduced operating expenses, and consistent fulfillment of performance objectives.

5 Experiment and Analysis

In the present study, the SRN model was implemented and analyzed using the Mercury Tool [61], a comprehensive probabilistic modeling and simulation environment designed for reliability and performance analysis. Additionally, Discrete Event Simulation (DES) methods were employed to offer deeper temporal granularity. DES is subject to variability across simulation runs. Therefore, for robust analysis, we conducted several simulations and analyzed the overall findings. As illustrated in Fig. 4, the SRN model captures the entirety of the S-iNAS system, adhering to the Default Input Parameters in Table 3 and the Guard Functions enumerated in Table 2 [6, 8]. These functions specify the precise criteria for activating timed or immediate transitions, thereby governing the intricate operational logic outlined in Sect. 3. All experiments were performed on a desktop system configured with *CPU:* Intel(R) Core(TM) i5-13400 *RAM:* 16 GB of DDR4 *Software Tool:* Mercury Tool for modeling and analysis Such a setup afforded sufficient computational resources for multiple simulation iterations and sensitivity analyses.

5.1 Default Input Parameters and Model Configuration

Table 3 details the system's Immediate/Timed Transitions, along with their parameter names, types, functional roles, default values, and units. Several key timed transitions, including $1/\lambda$, were derived from empirical observations acquired via the `rados bench` utility in Ceph, in line with prior live-environment evaluations [17]. The default $1/\lambda$ is set to 300 s, mirroring typical arrival rates.

Regarding write and replication processes, $1/\lambda_{write}$ and $1/\lambda_{replicate}$ were both assigned 0.0643413 s. This choice reflects the overhead incurred when writing and replicating 4 MB object blocks and is grounded in the default Ceph object size and established empirical insights on hybrid storage operations [8]. Conversely, $1/\lambda_{read}$ was set to 0.0185085 s, as read workflows do not entail replication overhead.

Infrastructure-level latencies were modeled by setting $1/\lambda_{vmcreationtime}$ and $1/\lambda_{vmdestroytime}$ to 30 s each, approximating standard durations for OSD instantiation or teardown under varying network conditions and data scales. Additionally, this parameter is designed so that scaling does not occur during VM provisioning due to the Scale Up and Down Guard Functions(gT^{sn}_{sud}, gT^{dsd}_{gn} in Table 4), thereby preventing any impact on the pre-allocated OSDs. The parameter $1/\lambda_{delayscaling}$, representing the length of time for data redistribution or CRUSH map updates during a scaling event, is initialized to 0 s—though nonzero values could be employed in production to forestall rapid oscillations in cluster sizing. Meanwhile, $1/\epsilon$ was fixed at 10 ms to account for common computational or network delays.

Besides P_{wr} and P_{rr}—probabilities for Write and Read operations, respectively—parameters N_{VC} and N_{PC} represent a node's virtual capacity and physical capacity. N_{OSD} governs the count of OSDs (or replicas) allocated to each node, resulting in two OSDs dedicated to writing and two to reading for a total of four per node. We set $N_{clients} = 8$ to model a moderately scaled environment, with each client generating 16 parallel threads ($TOC = 16$). Therefore, the network queue Q is computed as $N_{clients} \times TOC$. Lastly, S_RATE appears in the guard function gT^{sn}_{dsu} (see Table 2), determining the threshold level for triggering scaling transitions.

5.2 Performance Metrics: Throughput and Response Time

To comprehensively assess the S-iNAS model's performance, two performability metrics were evaluated [62]:

(1) Throughput: Measured as the cumulative volume of data processed (in MB/s) within a specific timeframe. Concretely, this quantity sums the number of tokens in the Write, Replicate, or Read states—namely, P^{si}_{w1o}, P^{si}_{w2o}, P^{si}_{rep1}, P^{si}_{rep2}, P^{si}_{r1}, and P^{si}_{r2}—each weighted by its corresponding transition rate. Formally:

$$\text{Throughput}^{si} = \sum_{i=1}^{n} \left(\frac{(\#P^{si}_{w1o} + \#P^{si}_{w2o})}{\lambda_{write}} + \frac{(\#P^{si}_{rep1} + \#P^{si}_{rep2})}{\lambda_{replicate}} + \frac{(\#P^{si}_{r1} + \#P^{si}_{r2})}{\lambda_{read}} \right) \quad (1)$$

where *n* is the total number of Storage Nodes under analysis.

(2) Response Time: Denoted $E[Snode^{si}_{job}]$, this metric equals the average count of jobs in the system at each Snode divided by the node's throughput. By defining Throughput^{si} according to Eq. (1):

$$\text{Response Time}^{si} = \sum_{i=1}^{n} \frac{E[Snode^{si}_{job}]}{\text{Throughput}^{si}} \quad (2)$$

Table 2 Guard functions & priority

Guard	Transition	Firing condition	Scale policy	Priority
gT^{sn}_{dsu}	T^{si}_{dsu}, T^{sj}_{dsu}	$(\#P^{sn}_{dsu} + \#P^{sn}_{i} + \#P^{sn}_{id} < 1) AND ((\#P^{sn}_{q} > 0) AND (\#P^{sn}_{pc} > 0) AND$ $((\#P^{sn}_{q}) > (\#P^{sn}_{c} + \#P^{sn}_{rs} + \#P^{sn}_{s1} + \#P^{sn}_{r1} + \#P^{sn}_{s2} + \#P^{sn}_{r2} + \#P^{sn}_{wr} + \#P^{sn}_{w1} + \#P^{sn}_{w1e}$ $+\#P^{sn}_{rep1} + \#P^{sn}_{rep1e} + \#P^{sn}_{w2} + \#P^{sn}_{w2e} + \#P^{sn}_{rep2} + \#P^{sn}_{rep2e} + \#P^{sn}_{a}) * S_RATE))$	Scale Up	1
gT^{sn}_{idi}	T^{si}_{idi}, T^{sj}_{idi}	$((\#P^{sn}_{w2} + \#P^{sn}_{w2o} + \#P^{sn}_{rep2} + \#P^{sn}_{w1} + \#P^{sn}_{w1o} + \#P^{sn}_{rep1} + \#P^{sn}_{r2} + \#P^{sn}_{r2o} + \#P^{sn}_{r1} + \#P^{sn}_{r1o}) = \#P^{sn}_{vc})$	Scale Up	3
gT^{sn}_{sud}	T^{si}_{sud}, T^{sj}_{sud}	$(\#P^{sn}_{id} = 1) AND ((\#P^{sn}_{c} + \#P^{sn}_{rs} + \#P^{sn}_{s1} + \#P^{sn}_{r1} + \#P^{sn}_{s2} + \#P^{sn}_{r2} + \#P^{sn}_{wr}$ $+\#P^{sn}_{w1} + \#P^{sn}_{w1e} + \#P^{sn}_{rep1} + \#P^{sn}_{rep1e} + \#P^{sn}_{w2} + \#P^{sn}_{w2e} + \#P^{sn}_{rep2} + \#P^{sn}_{rep2e} + \#pS1_{A}ck) < (\#P^{sn}_{vc}))$	Scale Up	4
gT^{sn}_{sud1}	T^{si}_{sud1}, T^{sj}_{sud1}	$(\#P^{sn}_{id} = 1) AND ((\#P^{sn}_{w2o} + \#P^{sn}_{rep2} + \#P^{sn}_{w2}) < (\#P^{sn}_{vc}/4))$	Scale Up	3
gT^{sn}_{sud2}	T^{si}_{sud2}, T^{sj}_{sud2}	$(\#P^{sn}_{id} = 1) AND ((\#P^{sn}_{w1o} + \#P^{sn}_{rep1} + \#P^{sn}_{w1}) < (\#P^{sn}_{vc}/4))$	Scale Up	3
gT^{sn}_{sud3}	T^{si}_{sud3}, T^{sj}_{sud3}	$(\#P^{sn}_{id} = 1) AND ((\#P^{sn}_{r2o} + \#P^{sn}_{r2}) < (\#P^{sn}_{vc}/4))$	Scale Up	3
gT^{sn}_{sud4}	T^{si}_{sud4}, T^{sj}_{sud4}	$(\#P^{sn}_{id} = 1) AND ((\#P^{sn}_{r1o} + \#P^{sn}_{r1}) < (\#P^{sn}_{vc}/4))$	Scale Up	3
gT^{sn}_{dsd}	T^{si}_{dsd}, T^{sj}_{dsd}	$(\#P^{sn}_{dsd} + \#P^{sn}_{r} + \#P^{sn}_{rd} < 1) AND ((\#P^{sn}_{vc} > 4) AND (\#P^{sn}_{q} < 1) AND (\#P^{sn}_{c}))$	Scale Down	1
gT^{sn}_{rdi}	T^{si}_{rdi}, T^{sj}_{rdi}	$(\#P^{sn}_{r} > 0) AND ((\#P^{sn}_{w2} + \#P^{sn}_{w2o} + \#P^{sn}_{rep2} + \#P^{sn}_{w1} + \#P^{sn}_{w1o} + \#P^{sn}_{rep1} +$ $\#P^{sn}_{r2} + \#P^{sn}_{r2o} + \#P^{sn}_{r1} + \#P^{sn}_{r1o}) = \#P^{sn}_{vc})$	Scale Down	3
gT^{sn}_{sdd}	T^{si}_{sdd}, T^{sj}_{sdd}	$((\#P^{sn}_{c} + \#P^{sn}_{rs} + \#P^{sn}_{s1} + \#P^{sn}_{r1} + \#P^{sn}_{s2} + \#P^{sn}_{r2} + \#P^{sn}_{wr} + \#P^{sn}_{w1}$ $+\#P^{sn}_{w1e} + \#P^{sn}_{rep1} + \#P^{sn}_{rep1e} + \#P^{sn}_{w2} + \#P^{sn}_{w2e} + \#P^{sn}_{rep2} + \#P^{sn}_{rep2e} + \#pS1_{A}ck) > (\#P^{sn}_{vc}))$	Scale Down	4
gT^{sn}_{sdd1}	T^{si}_{sdd1}, T^{sj}_{sdd1}	$(\#P^{sn}_{rd} = 1) AND ((\#P^{sn}_{w2o} + \#P^{sn}_{rep2} + \#P^{sn}_{w2}) > (\#P^{sn}_{vc}/4))$	Scale Down	3

Table 2 (continued)

Guard	Transition	Firing condition	Scale policy	Priority
gT^{sn}_{sdd2}	T^{si}_{sdd2}, T^{sj}_{sdd2}	$(\#P^{sn}_{rd} = 1)AND((\#P^{sn}_{w1o} + \#P^{sn}_{rep1} + \#P^{sn}_{w1}) > (\#P^{sn}_{vc}/4))$	Scale Down	3
gT^{sn}_{sdd3}	T^{si}_{sdd3}, T^{sj}_{sdd3}	$(\#P^{sn}_{rd} = 1)AND((\#P^{sn}_{r2o} + \#P^{sn}_{r2}) > (\#P^{sn}_{vc}/4))$	Scale Down	3
gT^{sn}_{sdd4}	T^{si}_{sdd4}, T^{sj}_{sdd4}	$(\#P^{sn}_{rd} = 1)AND((\#P^{sn}_{r1o} + \#P^{sn}_{r1}) > (\#P^{sn}_{vc}/4))$	Scale Down	3
gT^{si}_{m}	T^{si}_{m}	$(((\#P^{si}_{pc} < 1)AND(\#P^{sn}_{q} > \#P^{sn}_{c})AND(\#P^{si}_{dsu} + \#P^{si}_{i} + \#P^{sn}_{id} < 1)AND(\#P^{sj}_{stop} > 0))$ $OR((\#P^{si}_{pc} > 0)AND(\#P^{sj}_{q} < 1)AND(\#P^{sj}_{up} > 0)))$ $AND((\#P^{sij}_{c} + \#P^{sij}_{dr} + \#P^{sij}_{cso} + \#P^{sij}_{csi} + \#P^{sij}_{drso} + \#P^{sij}_{drsi} + \#P^{sij}_{sos} + \#P^{sij}_{sis}) < 1)$	Scale In/ Out	3
gT^{si}_{mc}	T^{si}_{mc}	$(\#P^{si}_{vc} >= \#P^{sj}_{vc})AND(\#P^{sj}_{stop} > 0)$	Scale Out	3
gT^{sj}_{mc}	T^{sj}_{mc}	$(\#P^{sj}_{pc} >= N_{pc})AND(\#P^{sj}_{up} > 0)$	Scale In	3
gT^{si}_{mdr}	T^{si}_{mdr}	$(\#P^{si}_{q} >= \#P^{sj}_{q})AND(\#P^{sj}_{stop} > 0)$	Scale Out	3
gT^{sj}_{mdr}	T^{sj}_{mdr}	$(\#P^{sj}_{q} < 1)AND(\#P^{sj}_{up} > 0)$	Scale In	3
Other transition informations		$T^{si}_{rept1}, T^{si}_{rept2}, T^{si}_{rep1d}, T^{si}_{rep2d}, T^{si}_{a}, T^{sj}_{rept1}, T^{sj}_{rept2}, T^{sj}_{rep1d}, T^{sj}_{rep2d}, T^{sj}_{a}$ have no guard functions and priority is 2		
		The remaining Transitions have no guardfunctions and priority is 1		

Table 3 Default input parameters

Parameters	Associated types	Description	Value	
$1/\lambda$	T_a	Mean time to a request arrival	10	s [17]
$1/\lambda_{write}$	$T^{si}_{w1}, T^{si}_{w2}, T^{sj}_{w1}, T^{sj}_{w2}$	Mean time to a writing	0.0643413	s [8]
$1/\lambda_{read}$	$T^{si}_{r1}, T^{si}_{r2}, T^{sj}_{r1}, T^{sj}_{r2}$	Mean time to a reading	0.0185085	s [8]
$1/\lambda_{replicate}$	$T^{si}_{rep1}, T^{si}_{rep2}, T^{sj}_{rep1}, T^{sj}_{rep2}$	Mean time to a replicating	0.0643413	s [8]
$1/\mu_{vmcreationtime}$	T^{si}_{id}, T^{sj}_{id}	Mean time to a Virtual Machine Creation Time	30	s
$1/\mu_{VMDestroyTime}$	T^{si}_{rd}, T^{sj}_{rd}	Mean time to a Virtual Machine Destroy Time	30	s
$1/\lambda_{DelayScaling}$	T^{si}_{ds}, T^{sj}_{ds}	Mean time to a Delay Scaling(Time based Policy)	0	s
$1/\epsilon$	$T_{sd}, T_{ss}, T_{roi}, T_{gcm}$ $T_{gpg}, T^{si}_{d}, T^{d}_{sj}, T^{si}_{w1s}$ $T^{si}_{w2s}, T^{sj}_{w1s}, T^{sj}_{w2s}, T^{si}_{s1}$ $T^{si}_{s2}, T^{sj}_{s1}, T^{sj}_{s2}$	Mean time to a general delay of processing or network	0.01	s [6]
P_{wr}	T^{si}_{wr}, T^{sj}_{wr}	Coverage Factor of write job distribution	50%	
P_{rr}	T^{si}_{rr}, T^{sj}_{rr}	Coverage Factor of read job distribution	50%	
N_{VC}	Definition	Virtual Capacity of Each Node	4	units
N_{PC}	Definition	Physical Capacity of Each Node	16	units
N_{OSD}	Definition	OSD Capacity of Each Node	1	unit
$N_{clients}$	Definition	The number of ceph clients	8	units [17]
TOC	Definition	The number of Threads per client	16	units [17]
Q	Definition	Network Queue Size according to number of clients	128	units [17]
S_RATE	Definition	Scaling policy default input parameter(Event based Policy) in a scaling rate of guard function	1	

yielding the mean time that newly arriving requests wait or spend in the system before completion.

5.3 Experiment Setup and Scaling Policies

This study create the model using the Mercury Tool. The model is generated using a script instead of simulating in a GUI environment due to its large size and the numerous parameters that needed to be modified. The script in MRY format is executed using Java to obtain the Stationary Simulation(warmup:30000000, confidenceLevel:99, maxRelativeError:0.01, batchSize:100). Exploiting the Mercury Tool, we applied the entire SRN model (Fig. 4) under two key Scaling Policies:

- *Time-based Scaling:* (Time60, Time120, Time180). These policies impose a fixed interval (60 s, 120 s, or 180 s) to prevent excessive resource scaling events. Time-Based is performed by changing the $1/\lambda_{DelayScaling}$ in Table 3.
- *Event-based Scaling:* (Event25, Event50, Event75). These policies activate scaling when the ratio of the average count of jobs to idle resources surpasses a specified threshold (25%, 50%, or 75%). Event-Based is performed by changing the S_RATE of gT^{sn}_{dsu} in Table 2.

Table 4 compiles results from both Stationary Analysis (under default assumptions) and Sensitivity Analysis (with certain parameters varied while others remain stable).

5.3.1 Stationary Analysis

We commenced by inspecting the system's steady-state behavior, deploying the Default Input Parameters in Table 3 for each scaling approach. Under the Time-based scenarios (Time60, Time120, Time180), scaling is deliberately spaced out to stave off instability from frequent adjustments. Conversely, the Event-based scenario (Event25, Event50, Event75) triggers expansions or contractions in response to real-time queue load relative to idle resources.

Table 4 and Fig. 8 highlights key performance metrics:

- `Throughput:`
  - Using default settings, baseline throughput is 115.84 MB/s.
  - The Time120 scenario delivers the lowest throughput (104.99 MB/s), suggesting delayed scaling or under-used resources.
  - Event25 and Event75 outstrip the default throughput (118.03 MB/s and 117.45 MB/s, respectively), indicating that threshold-based scaling adapts effectively to surging workloads.
- `Response Time:`
  - Under default conditions, average response time is 1.05 s.

Table 4 Stationary analysis under default input parameters

Metrics	Default	Time60	Time120	Time180	Event25	Event50	Event75
Throughput (MB/s)	115.84	107.86	104.99	107.67	118.03	117.45	117.50
Response Time (s)	1.05	1.48	1.31	1.86	1.17	1.28	1.14

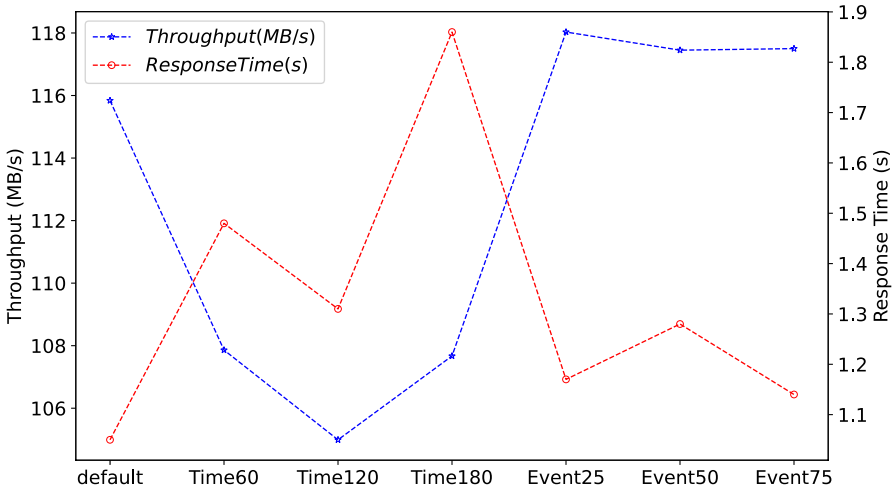


Fig. 8 Stationary analysis under throughput and response time of default input parameters

- Time180 shows the largest response time (1.86 s), likely reflecting deferred scaling under heavier loads.
- Event25 and Event75 reduce response time to 1.17 s and 1.14 s, respectively, pointing to quicker scaling actions and correspondingly shorter queue delays.

The reason why the time-based throughput is lower than the default parameter is that, despite having a large number of operations waiting in the queue compared to the available OSDs, scaling up is not possible, and the number of available OSDs does not increase by the DelayScaling time($\lambda_{DelayScaling}$ in Table 3). In contrast, the event-based policy allows the number of available OSDs to scale according to the number of operations waiting in the queue without the DelayScaling time, relative to the number of OSDs available. Therefore, while Time-based Scaling helps avoid constant re-scaling, it can sacrifice responsiveness under prolonged or rapid changes in demand. In contrast, Event-based Scaling is more agile, offering improved throughput and lower latency in the tested conditions.

By correlating these findings with practical workloads, administrators can adjust interval lengths or threshold values in pursuit of specific operational goals, for instance prioritizing minimal latency or maximizing cost-effectiveness. The subsequent sections detail additional Sensitivity Analyses of how variations in arrival rates, number of clients, VM instantiation time, and job composition (Write vs. Read) shape global system performance.

5.4 Sensitivity Analysis

To achieve a deeper and more comprehensive understanding of how the system model behaves under different Scaling Policies, we conducted a Sensitivity Analysis, examining four critical parameters that directly influence system performance:

1. Arrival Time (λ)
2. Number of Clients ($N_{clients}$)
3. VM Creation Time (μ)
4. Job Types (Write: P_{wr} vs. Read: P_{rr})

To keep the results tractable, we use the Default Input Parameters scenario as a baseline for all comparisons. Accordingly, each graphical result set is partitioned into two distinct plots: one contrasts the Default scenario with the Time-based Scaling Policy, and the other compares the Default scenario with the Event-based Scaling Policy.

Throughput Analysis. Fig. 9 encapsulates how throughput responds to variations in the aforementioned parameters.

- *Arrival Time* ($1/\lambda$): Fig. 9a and b present the system's throughput as arrival time increases. Under the Default setting, throughput declines proportionally with the lengthening of arrival time, implying that rising system load eventually diminishes throughput. When the Time-based Scaling Policy is engaged, a comparable downward trend emerges, but `Time60` and the Default scenario tend to produce the highest throughput values, whereas `Time180` manifests the lowest performance. This suggests that the 60-second interval effectively calibrates scaling events to handle increasing traffic.
  In contrast, Event-based Scaling introduces throughput variability contingent upon the interplay between available OSDs and queued requests. As the trigger threshold (25%, 50%, or 75%) is raised, the system endures heavier load before scaling, which can diminish throughput. Notably, `Event25` attains the highest throughput among the event-based policies, whereas `Event75` reports the lowest. This outcome indicates that a more proactive threshold (25%) manages mounting workloads more swiftly.
- *Number of Clients* ($N_{clients}$): Fig. 9c and d highlight how throughput evolves with an increasing number of Ceph clients. Under the Default scenario, throughput grows in tandem with client numbers, reflecting effective parallelization. In Time-based Scaling, throughput remains similar or slightly lower than Default for `Time60` but drops markedly in `Time120` and `Time180`, underscoring slower responsiveness to rises in load. Conversely, Event-based Scaling showcases `Event25` as the top performer, followed sequentially by `Event50`, `Event75`, and lastly, the Default scenario. Accordingly, a 25% threshold fosters more rapid adaptation, furnishing superior overall throughput.
- *VM Creation Time* ($1/\mu_{vmcreationtime}$): Fig. 9e and f depict the sensitivity of throughput to VM instantiation delays. Under the Default scenario, throughput gradually declines as VM creation time elongates. In the Time-based Scaling approach, `Time60` registers slightly higher throughput than Default; `Time120` and

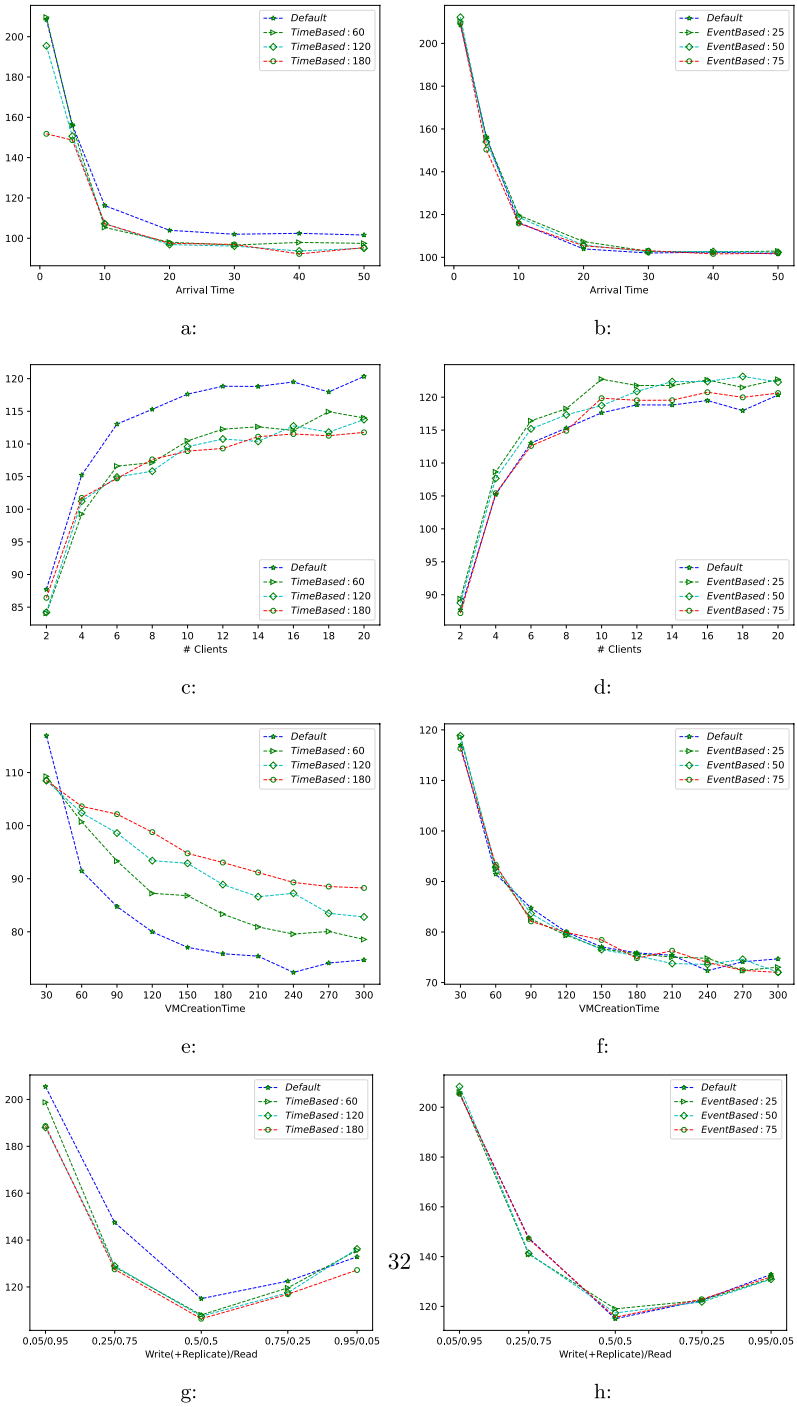


Fig. 9 Throughput with respect to sensitivity parameter. Y-axis: throughput (MB/s). X-axis: **a**, **b** arrival time, **c**, **d** number of clients, **e**, **f** VM creation time, **g**, **h** job types (write, replicate/read)

`Time180` display marginally lower outcomes. Meanwhile, Event-based Scaling remains close to the Default's performance trajectory, suggesting minimal impact from changing instantiation times. Here, `Event50` and `Event75` align tightly with the Default scenario, while `Event75` experiences a minor performance dip for extended creation times.

- *Write vs. Read Job Ratios* (P_{wr}/P_{rr}): Fig. 9g and h examine how throughput reacts to varying proportions of Write (P_{wr}) and Read (P_{rr}) requests. Under the Default configuration, throughput is maximized at a low write ratio (0.05), then declines at intermediate ratios (0.25 and 0.50), before slightly rebounding at higher write ratios (0.75 and 0.95). This pattern arises partly from replication overhead during write operations, which is needed to ensure high data availability.
  In the Event-based Scaling scenarios, throughput remains similar to or slightly exceeds the Default case, suggesting that quicker or more proactive scaling decisions offset the overhead of write-intensive workloads. By comparison, the Time-based Scaling approach tends to show somewhat lower throughput, illustrating that reactive scaling on fixed intervals can be less efficient under intensive write patterns.

Overall, these sensitivity experiments underscore the nuanced interplay between dynamic workload characteristics (arrival times, client populations, VM instantiation durations, and job-type distributions) and the chosen scaling mechanism (Time-based vs. Event-based). Although Time-based Scaling can mitigate over-scaling, it may occasionally lag in addressing surges in demand, thus impacting throughput adversely. By contrast, Event-based Scaling offers more agile responses, sustaining throughput at higher levels under changing conditions. Through meticulous calibration of these policies and parameters, system architects and administrators can more precisely tailor performance to specific operational goals, balancing cost, latency, and resource utilization requirements.

`Response Time`: Fig. 10 illustrates the results of the Sensitivity Analysis regarding system response time. We investigate how changes in arrival rate $(1/\lambda)$, the number of Ceph clients $(N_{clients})$, VM creation time $(1/\mu_{vmcreationtime})$, and job-type ratios (P_{wr}/P_{rr}) affect average response latency under both Time-based and Event-based Scaling Policies.

- *Arrival Time* $(1/\lambda)$: Fig. 10a and b depict how response time behaves as arrival time increases. Under the Default scenario, response time declines with longer arrival intervals, indicating enhanced system responsiveness as the instantaneous rate of job arrivals moderates. Among the Time-based Scaling configurations, `Time60` yields the shortest response time, whereas `Time180` exhibits the highest latency. Interestingly, the Default scenario outperforms all Time-based policies in overall responsiveness, suggesting that imposing a fixed scaling interval can introduce delays in adapting to the real-time workload.
  By contrast, Event-based Scaling demonstrates relatively stable response times, even when event thresholds (25%, 50%, or 75%) grow. Specifically, `Event25` consistently achieves the lowest response time, while `Event75` experiences a marginal uptick, yet remains fairly close to other event thresholds. This stability

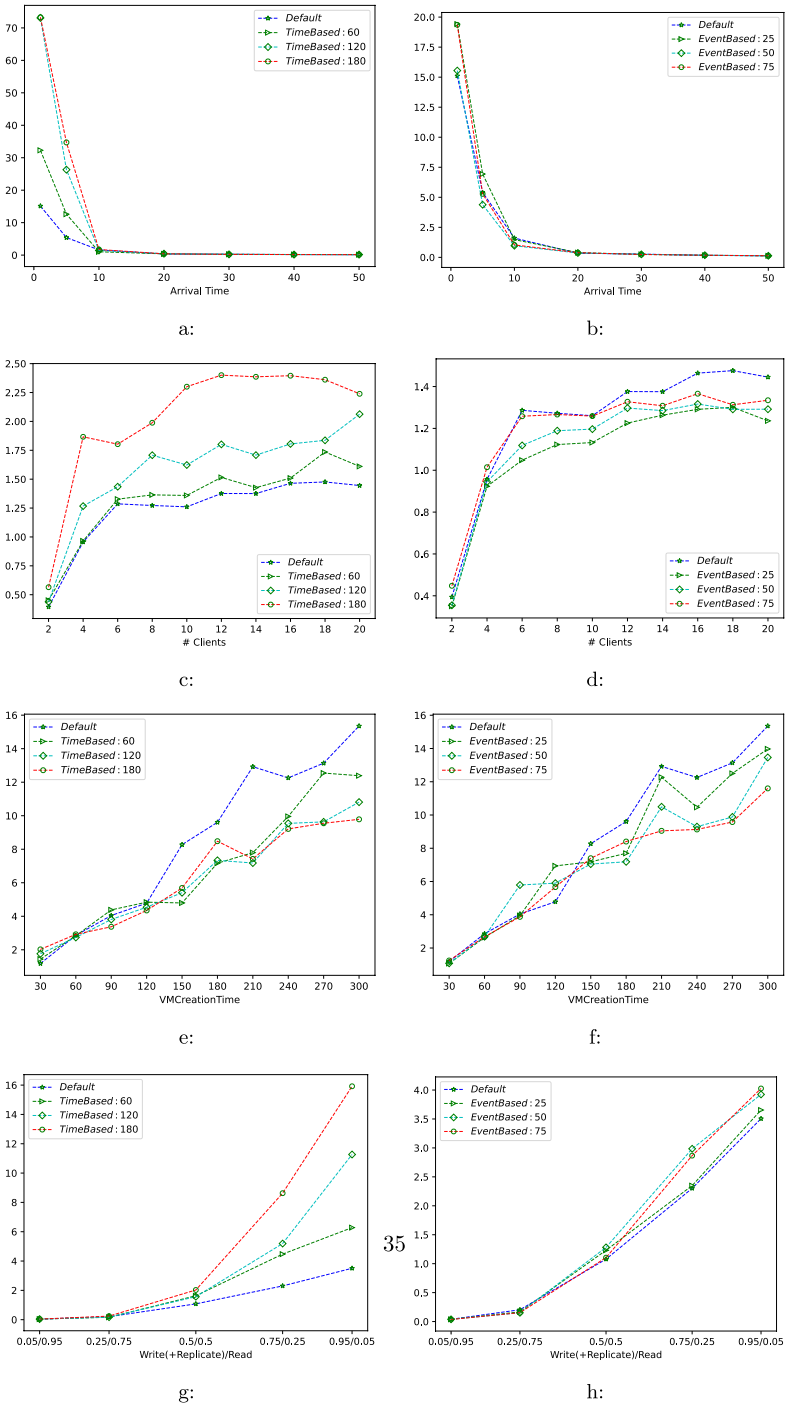


Fig. 10 Response time with respect to sensitivity parameter. Y-axis: response time(s), X-axis: **a**, **b** arrival time, **c**, **d** the number of clients, **e**, **f** VM creation time, **g**, **h**, job types (write/read)

evidences that Event-based Scaling is adept at preserving consistent responsiveness across varying workload conditions.

- *Number of Ceph Clients* ($N_{clients}$): Fig. 10c and d examine how response time evolves as the client population increases. In the Default scenario, latency escalates alongside the expanding client base, reflecting heightened contention for shared resources. Under the Time-based Scaling regime, `Time60`, `Time120`, and `Time180` all display higher response times than Default. Although `Time60` fares relatively better than the longer intervals, it does not surpass the Default configuration.
  Conversely, Event-based Scaling consistently produces shorter or at least comparable response times. Among the thresholds, `Event25` yields the lowest latency, followed successively by `Event50`, `Event75`, and then Default. This ranking signifies that smaller thresholds (i.e., 25%) trigger scaling more quickly, thus improving responsiveness.
- *VM Creation Time* ($1/\mu_{vmcreationtime}$): Fig. 10e and f depict the impact of VM creation latencies on response time. In the Default scenario, response time increases in tandem with VM instantiation delays. Time-based Scaling offers better response times than Default. Notably, `Time60` exhibits somewhat higher latency, whereas `Time120` and `Time180` witness incrementally lower response times in comparison.
  Under Event-based Scaling, all three event thresholds outperform Default. Paradoxically, `Event25` reports the longest response time among these thresholds, while `Event50` and `Event75` deliver improved outcomes. This indicates that an overly aggressive threshold (25%) may sometimes induce rapid scaling transitions that momentarily affect latency, whereas moderate thresholds (50% or 75%) strike a balance between reactivity and overhead.
- *Write/Read Job Ratios* (P_{wr}, P_{rr}): Fig. 10g and h detail how shifting from read-dominant to write-dominant workloads influences response time. Under Default, a rise in the proportion of write requests generally elevates latency. At minimal write ratios (0.05), the system achieves its fastest responses; conversely, higher write fractions (0.25, 0.5, 0.75, 0.95) progressively amplify response times, driven by replication overheads and heavier I/O contention.
  Between Time-based Scaling and Default, Default attains superior performance, with response times improving in the order of `Time60`, `Time120`, and `Time180`. Event-based Scaling, on the other hand, exhibits somewhat higher response times than Default, but it consistently outperforms the Time-based approach. Hence, while Default excels when write loads escalate, the Event-based method still ensures lower latency than Time-based policies for most tested write ratios.

5.5 S-iNAS Implementation

Implementation starts by segregating Ceph public and replication flows on dedicated twenty five gigabit links, removing head of line contention, so recovery traffic floods without blocking clients, then the operator runs `cephadm bootstrap` to plant one

monitor and one manager, the orchestrator reconcile further daemons across hosts thus placement remains aligned and health rechecked continually. With the cluster active the placement group autoscaler stays in auto, PG counts rise with OSD capacity, manual edits of `pg_num` or `pgp_num` vanish. The manager Prometheus module emits one second counters such as `ceph_osd_op_in_progress` and `ceph_pool_recovery_bytes`, a control loop reads those series, judges backfill pressure, and Alertmanager forwards alerts to a webhook or a KEDA scaler (Kubernetes based Event Driven Autoscaler) that converts queue length into horizontal pod targets, hence OSD pods or metal nodes appear exactly when the SRN model predicts benefit. During expansion the admin lifts `osd_max_backfills` while capping `osd_recovery_max_active`; once recovery almost finishes the paramaters restore, ending a cycle that aligns traffic layout, orchestration, data spread, telemetry, event driven scaling and safegaurd tuning, keeping throughput high, latency steady across changing workloads.

Foundation of the Cluster Choose public and private interfaces for client IO and replication so no head of line blocks arise, and deploy at least twenty five gigabit Ethernet where sensor streams crowd [63]. Deploy `cephadm` to bootstrap three mons and two mgrs, letting the orchestrator reconcile daemon placement [64]. At bootstrap set `osd_memory_target` to fit host memory, the orchestrator later tunes it [65].

Automated Capacity Expansion Enable the PG autoscaler; it keeps counts synced with pool load, avoiding manual `pgp_num` work [66, 67]. An Ansible role scans `ceph orch device ls` then calls `ceph orch apply osd`, new OSDs arrive within one minute, matching the thirty second delay assumed in the SRN model [68]. In containers the same rule lives in a Rook `CephCluster` resource, Rook maps block devices and advertises them [69].

Event Based Scaling Controller Prometheus exposes queue depth, throughput, and recovery progress each second [70, 71]. A `PrometheusRule` fires when `ceph_osd_op_in_progress` exceeds fifty percent of threads for twenty seconds, Alertmanager forwards to a small Python service that calls `ceph orch apply osd-all-available-devices`. Inside Kubernetes, KEDA uses the same query to scale the rook operator job which spins up OSD pods.

Time Based Scaling Option Where traffic follows shifts, cron tasks apply or mark out OSDs at fixed hours. A sleep equal to measured rebalance time prevents repeated loops; real clusters need fifteen to thirty minutes per ten terabytes when `osd_max_backfills` equals four and NVMe backs BlueStore DB.

Predictive Modelling Loop Encode the SRN in Mercury, translate each timed transition in Table 3 into a `.dpn`. Nightly CI sweeps arrival rates pulled from Prometheus JSON exporter, compares predicted queue length with dashboards [72]. If error rises beyond ten percent the controller widens or narrows the threshold and pushes a new rule to Prometheus.

Operational Safeguards Keep `osd_recovery_max_active` below half of `osd_max_backfills` to protect client IO [63]. Apply `bluestore_compression_algorithm lz4` only on archival pools to spare CPU during copy operations. Grafana Ceph dashboard helps operators confirm rebalance heat maps and ensure no drive holds above three percent of PGs [73].

Future Extensions Couple the event controller with a KEDA scaler that considers both queue depth and power budget, enabling green energy windows in edge factories. Integrate reinforcement learning after a month of telemetry, using SRN predicted reward as a safe baseline during exploration.

5.6 Discussion and Limitation

In this work, we introduce an SRN driven approach to analyse and predict Ceph perfomance. Measurements cover time based and event based scenarios, four parameters each. The analys focuses on throughput and response time of Write and Read operations. An input load of 1 GB is sliced into uniform 4 MB objects, enabling precise metric extraction.

To restrain initial complexity, the model isolates core data processing rather than mapping every Ceph detail, leading to two key limits. (i) *First*, infinite physical capacity is assumed, so the performance drop near saturation and the capacity triggered rebalance remains unseen. (ii) *Second*, a uniform load is applied. Enterprise clusters confront heterogeneous object sizes, from tiny logs through medium backup files to high resolution streaming content, as well as sequential and random accesses, hence the present resuls cannot generalise scaling guidance for such diversity. Extending the SRN with finite capacity and realistic traces would allow deeper analys of Ceph behvaiour and assist auto scaling policie.

The S-iNAS stduy targets elastic Ceph clusters under variable industrial demand. Its prime outcome is an SRN capturing concurrent I/O and replication overhead during scale out or scale up, partially filling the storage level autoscaling gap. However several management facets remain open. *First*, schedule based and threshold based triggers are modelled without the monitoring stack that feeds them, so policies can react slowly or oscillate when load shifts rapidly. *Second*, the network substrate is abstracted even though Li et al. [24] associate link capacity and latency with bottlenecks during expansion. *Third*, cost factors such as CPU, RAM and energy receive little attention, whereas Zouet al. [42] show their relevance to deployment choice. S-iNAS therefore offers a starting point for an integrated framework that cooperates with telemetry, SDN control and cost aware orchestration. Grounded in formal SRN theory, it complements heuristic and learning driven managers and could bridge analytic and operational perspectives.

Current Challenges and Future Directions Ceph rebalance traffic complicates accurate prediction because migration trafic can negate latency gains when thresholds are small. Energy demand and carbon footprint remain largely unquantified despite recent work on scaling policy reliability for 5 G services [74]. Model calibration relies on real time telemetry, urging integrated INT pipelines. Deep Q network controllers for service function chains reveal convergence delay in large state spaces [75], indicating that explainable reinforcement learning is required at the storage layer. Blending SRN exploration with learning agents could generate policy suggestions while keeping analytical transparency. Multi objective formulations covering energy, cost and recovery time would raise practical relevance. Cross layer orchestration that aligns transport routing, cache placement and OSD scaling promises end to end

optimisation. Finally, controlled field trials, similar to CDN selection experiments in [76], would validate assumptions under production interference.

6 Conclusion

This work introduced an SRN model tailored for S-iNAS systems, focusing on two principal Scaling Policies: Time-based and Event-based. The Time-based policy curtails frequent OSD scaling, whereas the Event-based approach adjusts resources in real-time by comparing the average count of jobs with idle capacity. After formulating and implementing the SRN model to incorporate these policies, we performed Sensitivity Analyses centered on vital parameters such as Arrival Rate, Number of Clients, VM Creation Time, and Job Types (Write/Read). The study evaluated crucial performance metrics including Throughput and Response Time. Key findings indicate that, under default conditions, throughput gradually diminishes as arrival time extends. Time-based Scaling often sustains comparatively moderate throughput but suffers pronounced response-time spikes when workloads intensify. By contrast, Event-based Scaling yields more stable throughput and can effectively mitigate latency, especially in use-cases featuring sudden or unpredictable demands. Analysis of varying client numbers and VM creation times further corroborates that Event-based approaches consistently adapt more swiftly to surging loads. For write-intensive workloads, Event-based Scaling can still achieve relatively higher performance gains than Time-based approaches, although Default parameters sometimes outperform both. Consequently, these insights can guide practitioners when deploying Ceph-based S-iNAS infrastructures or analogous scalable storage solutions. By forecasting workload characteristics—such as the proportion of write jobs, arrival rates, or the practical overhead of VM instantiation—designers can intelligently select or tune scaling policies. While Time-based Scaling bolsters throughput consistency, Event-based Scaling more effectively curbs response time, affording a refined trade-off between operational efficiency and resource usage. Time-based scaling adjusts resources at fixed intervals, providing predictability and consistency. Additionally, it is easy to manage due to its simple initial setup. In contrast, event-based scaling responds immediately to changes in state, reducing latency. It also optimizes resources by allocating them only when necessary, allowing for greater flexibility. It is expected that these methods can be appropriately applied depending on the specific use case.

Acknowledgements This research was partially supported by Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education (No. 2020R1A6A1A03046811). This research was partially supported by the MSIT (Ministry of Science, ICT), Korea, under the ITRC (Information Technology Research Centre) support program (IITP-2020-2016-0-00465) supervised by the IITP (Institute for Information & communications Technology Planning & Evaluation). This research was supported by Basic Science Research Program through the National Research Foundation of Korea(NRF) funded by the Ministry of Education (2021R1A2C2094943). This research was supported by Basic Science Research Program through the National Research Foundation of Korea(NRF) funded by the Ministry of Education (No. 2020R1A6A1A03046811) and (2021R1A2C2094943). This paper was supported by Korea Institute for Advancement of Technology (KIAT) grant funded by the Korea Government(MOTIE) (P0020536, HRD Program for Industrial Innovation). This work is financed through

national funds by FCT - Fundação para a Ciência e a Tecnologia, I.P., in the framework of the Project UIDB/00326/2025 and UIDP/00326/2025.

Author Contributions Conceptualization: T.A.N, D.M.; methodology: T.A.N.; software: D.L., J.L., E.C. and R.M.C.; validation: B.N. and M.T.; formal analysis: E.A. and G.C.; investigation: F.A.S. and P.M.; resources: D.M.; data curation: D.L.; writing-original draft preparation: D.L. and T.A.N; writing-review and editing: R.M.C, B.N., E.A., G.C., M.D.M., M.T., F.A.S., and P.M.; visualization: D.L. and J.L.; funding acquisition: D.M. and M.T.; All authors have read and agreed.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors have no relevant financial or non-financial interests to disclose. The authors have no Conflict of interest to declare that are relevant to the content of this article. All authors certify that they have no affiliations with or involvement in any organization or entity with any financial interest or non-financial interest in the subject matter or materials discussed in this manuscript. The authors have no financial or proprietary interests in any material discussed in this article.

References

1. Xu, L.D., Xu, E.L., Li, L.: Industry 4.0: state of the art and future trends. Int. J. Prod. Res. **56**(8), 2941–2962 (2018). https://doi.org/10.1080/00207543.2018.1444806
2. Yin, S., Kaynak, O.: Big data for modern industry: challenges and trends [point of view]. Proc. IEEE **103**(2), 143–146 (2015). https://doi.org/10.1109/JPROC.2015.2388958
3. Hermann, M., Pentek, T., Otto, B.: Design principles for industrie 4.0 scenarios. In 2016 49th Hawaii International Conference on System Sciences (HICSS), pp. 3928–3937. IEEE (2016)
4. Yu, W., Liang, F., He, X., Hatcher, W.G., Lu, C., Lin, J., Yang, X.: A survey on the edge computing for the internet of things. IEEE Access **6**, 6900–6919 (2018). https://doi.org/10.1109/ACCESS.2017.2778504
5. Badshah, A., Daud, A., Alharbey, R., Banjar, A., Bukhari, A., Alshemaimri, B.: Big data applications: overview, challenges and future. Artif. Intell. Rev. **57**(11), 290 (2024). https://doi.org/10.1007/s10462-024-10938-5
6. Weil, S.A., Brandt, S.A., Miller, E.L., Long, D.D.E., Maltzahn, C.: Ceph: a scalable, high-performance distributed file system. In: Proceedings of the 7th Symposium on Operating Systems Design and Implementation, OSDI '06, USA, pp. 307–320. USENIX Association (2006)
7. Gong, Z., Gu, X., Wilkes, J.: PRESS: predictive elastic resource scaling for cloud systems. In Proc. IFIP/IEEE Int'l Conference on Network and Service Management (CNSM), pp. 9–16 (2010)
8. Borba, E., Tavares, E.: dec. Stochastic modeling for performance and availability evaluation of hybrid storage systems. J. Syst. Softw. **134**, 1–11 (2017). https://doi.org/10.1016/j.jss.2017.08.043
9. Chattaraj, D., Sarma, M., Samanta, D.: Stochastic Petri Net Based Modeling for Analyzing Dependability of Big Data Storage System, pp. 473–484 (2019). https://doi.org/10.1007/978-981-13-1498-8_42
10. Chen, Z., Li, H., Ota, K., Dong, M.: HyScaler: a dynamic, hybrid VNF scaling system for building elastic service function chains across multiple servers. IEEE Trans. Netw. Serv. Manag. **20**(4), 4803–4814 (2023). https://doi.org/10.1109/TNSM.2023.3277552
11. Chouliaras, S., Sotiriadis, S.: nov. An adaptive auto-scaling framework for cloud resource provisioning. Future Gener. Comput. Syst. **148**, 173–183 (2023). https://doi.org/10.1016/j.future.2023.05.017
12. Evangelidis, A., Parker, D., Bahsoon, R.: Performance modelling and verification of cloud-based auto-scaling policies. Future Gener. Comput. Syst. **87**, 629–638 (2018). https://doi.org/10.1016/j.future.2017.12.047
13. Fé, I., Matos, R., Dantas, J., Melo, C., Nguyen, T.A., Min, D., Choi, E., Silva, F.A., Maciel, P.R.M.: Performance-cost trade-off in auto-scaling mechanisms for cloud computing. Sensors **22**(3), 1221 (2022). https://doi.org/10.3390/s22031221

14. Fortino, G., Guerrieri, A., Russo, W., Savaglio, C.: Integration of agent-based and cloud computing for the smart objects-oriented IoT. In: Proceedings of the 2014 IEEE 18th International Conference on Computer Supported Cooperative Work in Design (CSCWD), pp. 493–498. IEEE (2014)
15. Ganzhur, M., Dyachenko, N., Gazizov, A., Otakulov, A., Romanov, D.: Modeling of storage processes using Petri nets. E3S Web Conf. **175**, 05038 (2020). https://doi.org/10.1051/e3sconf/202017505038
16. Ghosh, R., Longo, F., Naik, V.K., Trivedi, K.S.: Modeling and Performance Analysis of Large Scale IaaS Clouds (2012)
17. Gudu, D., Hardt, M., Streit, A.: Evaluating the performance and scalability of the Ceph distributed storage system. In 2014 IEEE International Conference on Big Data, pp. 177–182 (2014)
18. Hance, T., Lattuada, A., Hawblitzel, C., Howell, J., Johnson, R., Parno, B.: Storage systems are distributed systems (so verify them that Way!). In: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), pp. 99–115. USENIX Association (2020)
19. Hristoski, I., Dimovski, T.: Petri net-based model of peer-to-peer dataset replication in big data. In Proceedings of the 3rd Virtual International Conference "Path to a Knowledge Society-Managing Risks and Innovation"(PaKSoM 2021), pp. 191–198 (2021). Complex System Research Center, Niš, Serbia; Mathematical Institute of the ...
20. Hyun, J., Van Tu, N., Yoo, J.H., Hong, J.W.K.: Real-time and fine-grained network monitoring using in-band network telemetry. Int. J. Netw. Manag. **29**(6), e2080 (2019). https://doi.org/10.1002/nem.2080
21. Jennings, B., Stadler, R.: Resource management in clouds: survey and research challenges. J. Netw. Syst. Manag. **23**(3), 567–619 (2015). https://doi.org/10.1007/s10922-014-9307-7
22. Karaagac, A., De Poorter, E., Hoebeke, J.: In-band network telemetry in industrial wireless sensor networks. IEEE Trans. Netw. Serv. Manag. **17**(1), 517–531 (2020). https://doi.org/10.1109/TNSM.2019.2949509
23. Li, Y., Lee, P.P., Lui, J.C.: Stochastic modeling of large-scale solid-state storage systems: analysis, design tradeoffs and optimization. SIGMETRICS Perform. Eval. Rev. **41**(1), 179–190 (2013). https://doi.org/10.1145/2494232.2465546
24. Li, Z., Wang, Y.: An adaptive read write optimized algorithm for Ceph heterogeneous systems via performance prediction and multi-attribute decision making. Clust. Comput. **26**, 1125–1146 (2023). https://doi.org/10.1007/s10586-022-03764-3
25. Kong, L.W., Moreno, O.: Characterization and prediction of performance loss and mttr during fault recovery on scale-out storage using doe & rsm: a case study with ceph. In: IEEE Transactions on Cloud Computing (2018)
26. Gudu, D., Hardt, M., Streit, A.: Evaluating the performance and scalability of the ceph distributed storage system. In: Proc. IEEE/ACM Cluster (2022)
27. Singh, S.T., Tiwari, M., Dhar, A.S.: Machine learning based workload prediction for auto-scaling cloud applications. In Proc, OTCON (2023)
28. Gao, J., Wang, H., Shen, H.: Machine learning based workload prediction in cloud computing. In Proc, IEEE ICCCN (2020)
29. Suleiman, B., Alibasa, M.J., Chang, Y.Y., Anaissi, A.: Predictive auto-scaling: Lstm based multi-step cloud workload prediction. In: Service-Oriented Computing - ICSOC Workshops, LNCS, pp. 5–16 (2024)
30. van der Ster, D., Wiebalck, A.: Enabling storage business continuity and disaster recovery with Ceph distributed storage. In Proc, CHEP (2023)
31. Asadi, A.N., Azgomi, M.A., Entezari-Maleki, R.: Unified power and performance analysis of cloud computing infrastructure using stochastic reward nets. Comput. Commun. **138**, 67–80 (2019)
32. Shojaee, R., Latifi, A., Yazdani, N.: A stochastic reward net approach to model availability of cloud virtualization. In Proc, IEEE IST (2014)
33. Trivedi, K.S., Bobbio, A.: Reliability and Availability Engineering: Modeling, Analysis, and Applications, 1st edn. Cambridge University Press, Cambridge (2017)
34. Merkouche, S., Bouanaka, C., Benkhelifa, E.: A petri net-based formal modeling for microservices auto-scaling. In: 2023 20th ACS/IEEE International Conference on Computer Systems and Applications (AICCSA), pp. 1–8. IEEE (2023)
35. Lorido-Botran, T., Miguel-Alonso, J., Lozano, J.A.: A review of auto-scaling techniques for elastic applications in cloud environments. J. Grid Comput. **12**(4), 559–592 (2014). https://doi.org/10.1007/s10723-014-9314-7
36. Mao, M., Li, J., Humphrey, M.: Cloud auto-scaling with deadline and budget constraints. In: 2010 11th IEEE/ACM International Conference on Grid Computing, pp. 41–48. IEEE (2010)

37. Muppala, J., Ciardo, G., Trivedi, K.S.: Stochastic reward nets for reliability prediction. Commun. Reliab., Maintain. Serviceability **1**(2), 9–20 (1994)
38. Van der Aalst, W.M., Van Hee, K.M., Reijers, H.A.: Analysis of discrete-time stochastic petri nets. Stat. Neerl. **54**(2), 237–255 (2000)
39. Dotoli, M., Fanti, M.: A generalized stochastic petri net model for management of distributed manufacturing systems. In Proceedings of the 44th IEEE Conference on Decision and Control, pp. 2125–2130. IEEE (2005)
40. Torres, E., Callou, G., Andrade, E.: A hierarchical approach for availability and performance analysis of private cloud storage services. Computing **100**(6), 621–644 (2018). https://doi.org/10.1007/s00607-018-0588-7
41. Borba, E., Tavares, E., Maciel, P.: sep. A modeling approach for estimating performance and energy consumption of storage systems. J. Comput. Syst. Sci. **128**, 86–106 (2022). https://doi.org/10.1016/j.jcss.2022.04.001
42. Zou, L., Rui, X., Nguyen, T.A., Min, D., Choi, E., Thang, T.D., Son, N.N.: A scalable network area storage with virtualization: modelling and evaluation using stochastic reward nets. In: Proceedings of the 1st International Conference on Information Science and Systems, ICISS '18, New York, NY, USA, pp. 225–233. Association for Computing Machinery (2018)
43. Shi, Y., Dong, M., Zhang, W., Liu, L., Zheng, Y., Cui, L., Zhang, J.: AdaptScale: An adaptive data scaling controller for improving the multiple performance requirements in Clouds. Future Gener. Comput. Syst. **105**, 814–823 (2020). https://doi.org/10.1016/j.future.2017.08.034
44. Gomes, C., Tavares, E., Junior, M.N.D.O., Nogueira, B.: Cloud storage availability and performance assessment: a study based on NoSQL DBMS. J. Supercomput. **78**(2), 2819–2839 (2022). https://doi.org/10.1007/s11227-021-03976-1
45. Alidoost Alanagh, Y., Firouzi, M., Rasouli Kenari, A., Shamsi, M.: Introducing an adaptive model for auto? Scaling cloud computing based on workload classification. 35 (2023)
46. Silva, A., Callou, G.: Models for availability evaluation of file servers in private clouds. Computing **107**(1), 11 (2025). https://doi.org/10.1007/s00607-024-01372-0
47. Sliem, M., Salmi, N., Ioualalen, M.: Performance analysis of self-adaptive policies in containerized microservices. In: 2021 International Conference on Engineering and Emerging Technologies (ICEET), pp. 1–6. IEEE (2021)
48. Mohamed, S., Forshaw, M., Thomas, N., Dinn, A.: Performance and dependability evaluation of distributed event-based systems: a dynamic code-injection approach. In Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering, ICPE '17, New York, NY, USA, pp. 349–352. Association for Computing Machinery (2017)
49. Jimenez, J., Soto, P., De Vleeschauwer, D., Chang, C.Y., De Bock, Y., Latré, S., Camelo, M.: Resource allocation of multi-user workloads in cloud and edge data-centers using reinforcement learning. In: 19th IFIP/IEEE Int. Conf. on Network and Service Management (CNSM), Niagara Falls, Canada, pp. 1–5. IEEE (2023)
50. Cui, Y., Zhang, S., Liu, Y.: Cedule+: Resource management for burstable cloud instances. IEEE Trans. Netw. Serv. Manag. (2020). https://doi.org/10.1109/TNSM.2020.3039942
51. Team, C.D.: Placement group autoscaler in Ceph nautilus (2019). https://docs.ceph.com/en/latest/rados/operations/placement-groups/#pg-autoscaler
52. van der Hooft, J., Claeys, M., Bouten, N., Wauters, T., Schönwälder, J., Pras, A., Stiller, B., Charalambides, M., Badonnel, R., Serrat, J., dos Santos, C.R.P., Turck, F.D.: Updated taxonomy for the network and service management research field. J. Netw. Syst. Manag. **26**(3), 790–808 (2018)
53. Marquezan, C.C., Bruneo, D., Longo, F., Wessling, F., Metzger, A., Puliafito, A.: 3-D cloud monitoring: enabling effective cloud infrastructure and application management. In: Proc. IFIP/IEEE Int'l Conference on Network and Service Management (CNSM), pp. 55–63 (2014)
54. Ko, Y.S., Park, J.H.: Placement of virtual network functions for network services. Int. J. Netw. Manag. (2021). https://doi.org/10.1002/nem.2232
55. Toth, B., Antal, P.: Libra: Adaptive pod scaling for kubernetes. In: IEEE/IFIP Network Operations and Management Symposium (2020)
56. Nguyen, V.G., Grinnemo, K.J., Taheri, J., Brunstrom, A.: On auto scaling and load balancing for user plane gateways in a softwarized 5g network. In: International Conference on Network and Service Management (2021)
57. Torquato, M., Maciel, P.R.M., Vieira, M.: Model-based performability and dependability evaluation of a system with VM migration as rejuvenation in the presence of bursty workloads. J. Netw. Syst. Manag. **30**(1), 3 (2022). https://doi.org/10.1007/s10922-021-09619-3

58. Zhang, H., Yuan, T.: Factored reinforcement learning for auto scaling in tandem queues. In: IEEE/IFIP Network Operations and Management Symposium (2022)
59. by macrovector / Freepik, D. Yellow grey conveyor system with control panel, robotic hands and packaging on line isometric vector illustration
60. Ciardo, G., Trivedi, K.S.: A decomposition approach for stochastic reward net models (1993)
61. Maciel, P., Matos, R., Silva, B., Figueiredo, J., Oliveira, D., Fé, I., Maciel, R., Dantas, J.: Mercury: Performance and dependability evaluation of systems with exponential, expolynomial, and general distributions. In: 2017 IEEE 22nd Pacific Rim international symposium on dependable computing (PRDC), 50–57. IEEE (2017)
62. Nguyen, T.A., Kim, M., Lee, J., Min, D., Lee, J.W., Kim, D.: Performability evaluation of switch-over moving target defense mechanisms in software-defined networking using stochastic reward nets. J. Netw. Comput. Appl. **199**, 103267 (2022). https://doi.org/10.1016/j.jnca.2021.103267
63. Weil, S.A., Brandt, S.A., Miller, E., Maltzahn, C.: CRUSH: Controlled, scalable, decentralized placement of replicated data. In: Proceedings of the 2006 ACM/IEEE Conference on Supercomputing (SC '06), 31 (2006)
64. Attardi, G., Barchiesi, A., Colla, A., Galeazzi, F., Marzulli, G., Reale, M.: Declarative modeling for building a cloud federation and cloud applications. In: Proceedings of the Cloud Federation Workshop (2017)
65. Weil, S.A., Leung, A.W., Brandt, S.A., Maltzahn, C.: RADOS: A scalable, reliable storage service for petabyte-scale storage clusters. In: 2nd IEEE/ACM International Workshop on Petascale Data Storage (PDSW '07), pp. 35–44 (2007)
66. Gudu, D., Hardt, M., Streit, A.: Evaluating the performance and scalability of the Ceph distributed storage system. In: Proceedings of the 2014 IEEE International Conference on Big Data, pp. 177–182 (2014)
67. Peters, A.J., van der Ster, D.C.: Evaluating CephFS performance versus cost on high-density commodity hardware. Comput. Softw. Big Sci. **5**, 25 (2021). https://doi.org/10.1007/s41781-021-00071-1
68. van der Ster, D.C., Lamanna, M., Mascetti, L., Peters, A.J., Rousseau, H.: Ceph-based storage services for run2 and beyond. J. Phys.: Conf. Ser. **664**, 042054 (2015). https://doi.org/10.1088/1742-6596/664/4/042054
69. Baumann, L., Militano, L., Bohnert, T.M., Benz, S.: Monitoring resilience in a Rook-managed containerized cloud storage system. In: 2019 European Conference on Networks and Communications (EuCNC), pp. 1–5 (2019)
70. Jones, R., Doidge, M., Hand, G., Love, P., Simpson, S.: A blueprint for a contemporary storage element, building a new WLCG storage system with widely available hardware and software components: Ceph, XRootD, and Prometheus. EPJ Web Conf. **295**, 01002 (2024)
71. Simili, E., Stewart, G., Roy, G., Skipsey, S., Britton, D.: A hybrid system for monitoring and automated recovery at the glasgow Tier-2 cluster. In EPJ Web Conf. **251**, 02047 (2021)
72. Pinheiro, T., Oliveira, D., Matos, R., Silva, B., Pereira, P., Melo, C., Oliveira, F., Tavares, E., Dantas, J., Maciel, P.: The Mercury environment: a modeling tool for performance and dependability evaluation. Intelligent Environments 2021. Ambient Intell. Smart Environ. **29**, 16–25 (2021). https://doi.org/10.3233/AISE210075
73. Jelten, J., Wollek, A., Frank, D., Lasser, T.: Equilibrium: optimization of Ceph cluster storage by size-aware shard balancing. (2023). arXiv preprint. https://doi.org/10.48550/arXiv.2310.15805
74. Perez, P., Mota, E.: Analysis of scaling policies for nfv providing 5g reliability requirements. IEEE Trans. Netw. Serv. Manag. (2022). https://doi.org/10.1109/TNSM.2022.3147146
75. Xiao, L., Wu, D.: Deep q networks based auto scaling for service function chaining. IEEE Trans. Netw. Serv. Manag. (2023). https://doi.org/10.1109/TNSM.2023.3333875
76. Silva, S.D., Queyreix, J.B.: A multiple source streaming solution enhancing consumer quality of experience. Int. J. Netw. Manag. (2020). https://doi.org/10.1002/nem.2081

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Damsub Lim received the M.S. and Ph.D. degrees in Computer science and Engineering fromKonkuk University, South Korea, in 2015 and 2025. He is a Postdoctoral researcher in theDepartment of Computer science and Engineering at Konkuk University. His research interestsinclude Distributed System, Cloud Computing, IoT and SDN, System Analysis and Modeling,Performance, Availability and Dependability Evaluation.

Tuan Anh Nguyen (Ph.D.'15, M.Sc.'10, B.Eng.'08) received his Ph.D. in Computer Scienceand Systems Engineering from Korea Aerospace University and his MSc and BEng inMechatronics from Hanoi University of Science and Technology. He is a member of IEEE, IEEEComputer, Robotics and Automation (IEEE RAS), Aerospace and Electronic Systems (IEEEAESS), and Reliability Societies. His research focuses on Deep Learning and DeepReinforcement Learning, AI-based Digital Twin Systems, Autonomous Intelligent Systems, andDepenability and Fault-Tolerance of Computer Systems and Networks.

Jueying Li received the M.S. and Ph.D. degrees in Computer science and Engineering fromKonkuk University, South Korea, in 2015 and 2025. She is a Postdoctoral researcher in theKonkuk Aerospace Design·Analysis Instititue at Konkuk University. Her research interestsinclude Cloud Resource Scheduling, Performability Analysis, AI Cloud and Cloud Architecture.

Eunmi Choi currently at Kookmin University, Korea, specialises in big data infrastructure,cloud computing, intelligent systems, information security, parallel and distributed systems, andsoftware architecture. She earned her M.S. and Ph.D. in Computer Science from Michigan StateUniversity (1991, 1997) and her B.S. from Korea University (1988). Previously, she was anassistant professor at Handong University (1998-2004). She leads the Distributed InformationSystem and Cloud Computing Lab at Kookmin University.

Dugki Min received a B.S. degree in industrial engineering from Korea University in 1986, anM.S. degree in 1991 and a Ph.D. degree in 1995, both in computer science from Michigan StateUniversity. He is currently a Professor in Department of Computer Science and Engineering andDepartment of Artificial Intelligent (Graduate School) at Konkuk University. His researchinterests include Cloud and Fog computing, Distributed and Parallel Computing, Deep Learningand Deep Reinforcement Learning, Intelligent Multi-Agent Systems, Software Architecture.

Ruben Machucho Cadena graduated as Electronic Engineer from the Institute of Technology ofYucatán in 1997. He gained his Master's degree in 2005 and Ph.D. degree in 2010, in ElectronicEngineering from Center of Research and Advances Studies (CINVESTAV) CampusGuadalajara, México. His research interests include pattern recognition, medical imageprocessing, robotics and applications of geometric algebra computing.

Bruno Nogueira received the Ph.D. degree in computer science from the Federal University ofPernambuco, in 2015. He is an Associate Professor with the Computing Institute, FederalUniversity of Alagoas, Brazil. His research interests include real-time and embedded systems,optimization, and modeling.

Ermeson Andrade is an associate professor at the Department of Computing at UniversidadeFederal Rural de Pernambuco (UFRPE), Brazil. From 2015 to 2016, he was a PostdoctoralFellow at VU University Amsterdam, Netherlands. In March 2014, he finished his Ph.D. degreein Computer Science at Informatics Center at Universidade Federal de Pernambuco (Cin-UFPE).From 2010 to 2011, he was a visiting scholar at the Department of Electrical and ComputerEngineering at Duke University, USA, under the supervision of Dr. Kishor Trivedi. He also hasover 10 years of experience in the industry, working with test and quality assurance.

Gustavo Callou received the PhD degree in computer science from the Federal University ofPernambuco, in 2013. He is an associate professor with the Department of Computing, FederalRural University of Pernambuco, Brazil. His research interest is focused in performance anddependency modeling, cloud computing, data centers and sustainability.

Mario Di Mauro is an assistant professor of Telecommunications at the University of Salerno,Italy. Before joining the academia, from 2007 to 2012 he was with Research Consortium onTelecommunications (formerly Ericsson Lab Italy) as a Team Leader in industrial research. Hismain fields of interest include: network performance, network security and availabilitycharacterization, data analysis for novel telecommunication infrastructures. He authored morethan 70 papers, mainly in the fields of network availability and security.

Matheus Torquato (Member, IEEE) received the master's degree in computer science from theFederal University of Pernambuco. He is currently pursuing the Ph.D. degree in informaticsengineering with the University of Coimbra. He is an Assistant Professor with the FederalInstitute of Alagoas, Campus Arapiraca. His current research focuses in the design anddevelopment of analytical models to evaluate performance, dependability, and security ofmoving target defense deployments in cloud computing. His research interests include cloudcomputing, performance, dependability, and security modeling.

Francisco Airton Silva received the B.S. degree in information systems from Instituto Federaldo Piauí (IFPI), in 2008, and the M.S. and Ph.D. degrees in computer science from UniversidadeFederal de Pernambuco (UFPE), in 2013 and 2017, respectively. In 2015, he studied withSapienza Universitã di Roma. He is currently a Professor of information systems withUniversidade Federal do Piauí (UFPI). He was a Collaborative Researcher in multinationaltechnology companies, such as EMC and OKI. His research interests include distributed systems,with an emphasis on cloud-fog integration, mobile computing, and systems performanceevaluation techniques.

Paulo Maciel received the degree in electronic engineering in 1987 and the M.Sc. and Ph.D.degrees in electronic engineering and computer science from the Federal University ofPernambuco, Brazil, respectively. He was a faculty member with the Department of ElectricalEngineering, Pernambuco State aUniversity, Brazil, from 1989 to 2003. Since 2001, he has beena member of the Informatics Center, Federal University of Pernambuco (UFPE), where he iscurrently an Associate Professor. In 2011, during his sabbatical from UFPE, he stayed with theDepartment of Electrical and Computer Engineering, Edmund T. Pratt School of Engineering,Duke University, Durham, NC, USA, as a visiting professor. His current research interestsinclude performance and dependability evaluation, Petri nets and formal models, encompassingmanufacturing, embedded, computational, and communication systems as well as powerconsumption analysis.

Authors and Affiliations

Damsub Lim[1] · Tuan Anh Nguyen[2] · Jueying Li[1] · Eunmi Choi[3] · Dugki Min[1,4] · Ruben Machucho Cadena[5] · Bruno Nogueira[6] · Ermeson Andrade[7] · Gustavo Callou[7] · Mario Di Mauro[8] · Matheus Torquato[9,10] · Francisco Airton Silva[11] · Paulo Maciel[12]

✉ Tuan Anh Nguyen
anhngt@huit.edu.vn

Damsub Lim
miaow89@konkuk.ac.kr

Jueying Li
lijueying1108@konkuk.ac.kr

Eunmi Choi
eunmi.choi@kookmin.ac.kr

Dugki Min
dkmin@konkuk.ac.kr

Ruben Machucho Cadena
rmachuchoc@upv.edu.mx

Bruno Nogueira
bruno@ic.ufal.br

Ermeson Andrade
ermeson.andrade@ufrpe.br

Gustavo Callou
gustavo.callou@ufrpe.br

Mario Di Mauro
mdimauro@unisa.it

Matheus Torquato
mtorquato@dei.uc.pt; matheus.torquato@ifal.edu.br

Francisco Airton Silva
faps@ufpi.edu.br

Paulo Maciel
prmm@cin.ufpe.br

[1] Department of Computer Science and Engineering, College of Engineering, Konkuk University, Seoul 05029, South Korea

[2] Department of Software Engineering, Faculty of Information Technology, Ho Chi Minh City University of Industry and Trade, Ho Chi Minh City 70000, Vietnam

[3] School of Software, College of Computer Science, Kookmin University, Seoul 02707, South Korea

[4] Department of Artificial Intelligence, Graduate School, Konkuk University, Seoul 05029, South Korea

[5] Information Technology Department, Polytechnic University of Victoria, 87138 Ciudad Victoria, Tamaulipas, Mexico

[6] Instituto de Computação, Universidade Federal de Pernambuco, Maceió, Alagoas, Brazil

[7] Department of Computing, Universidade Federal Rural de Pernambuco, Recife, Pernambuco 52171-900, Brazil

[8] Department of Information Engineering, Electrical Engineering and Applied Mathematics (DIEM), University of Salerno, 84084 Salerno, Italy

[9] University of Coimbra, CISUC/LASI, DEI, Coimbra, Portugal

[10] Instituto Federal de Alagoas (IFAL), Scalable Campus Arapiraca, Arapiraca, Brazil

[11] Laboratory of Applied Research to Distributed Systems (PASID), Federal University of Piauí (UFPI), Picos, Piauí, Brazil

[12] MoDCS Research Group, Centro de Informática, Universidade Federal de Pernambuco, Recife, Pernambuco, Brazil